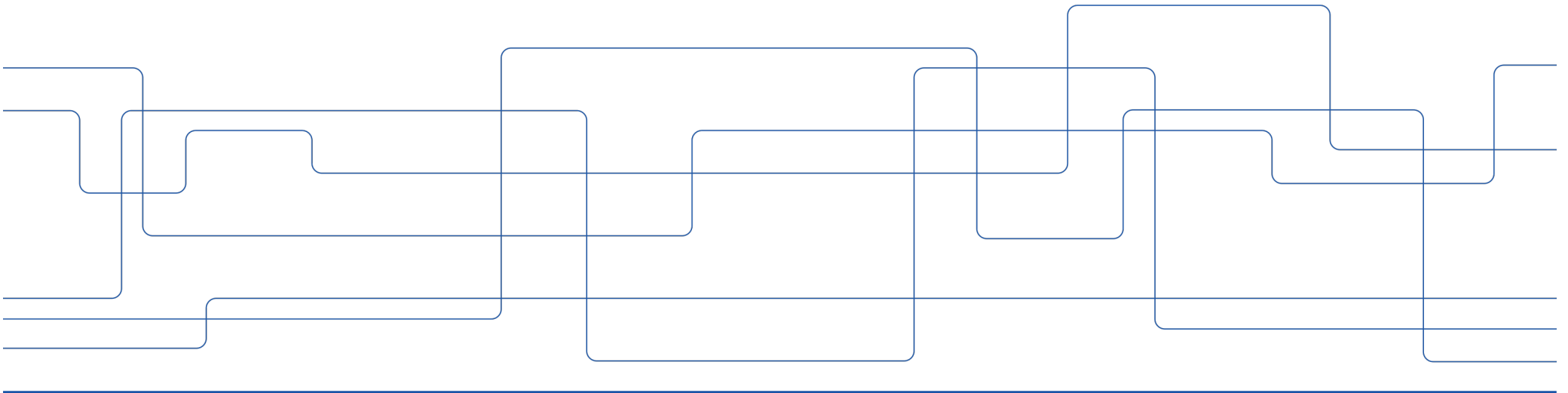




Lecture 8: Behavior Trees and Task Switching

by Petter Ögren



Question: Who has a plan? Everyone! ...has multiple plans...

Goal Directed

Reactive



Classical AI Plan
(High level)



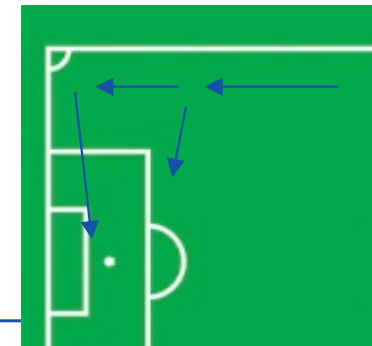
- Go to house
- Ring doorbell
- Deliver
 - to person
 - at door
- Repeat



- Disassemble tire
- Find leak
- Fix
 - plug leak
 - replace hose
- Assemble



Classical Control Policy
(Low level)





Everyone has a Plan/Policy

Classical AI
Plan
(High level)



Classical
Control Policy
(Low level)



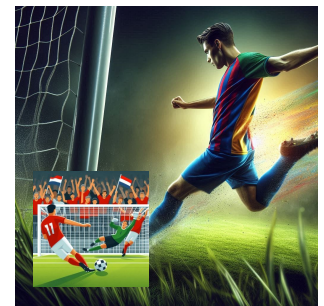
- Can we merge High level – Low level?
- A Plan that is a Policy (reactive)
 - A Policy that is a Plan (goal directed)



What we want in a Plan/Policy

- Reactive & Goal Directed
 - Handles positive and negative surprises
- Hierarchical
 - Combine policies to form new policy
- Modular
 - Can reuse policies
 - Few dependencies
- Can include
 - Classical Control design
 - Reinforcement Learning
- Can Guarantee Overall performance
 - Safe
 - Will reach goal

Behavior Trees





Content

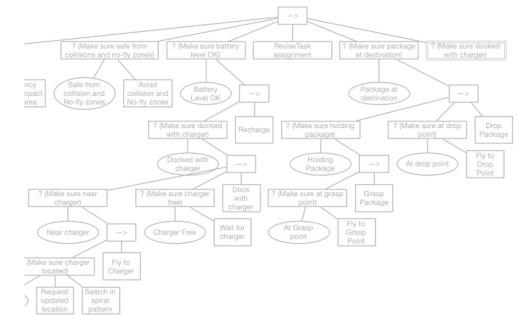
- What is a Behavior Tree?
 - A highly modular hierarchical combination of sub-controllers
 - When/Why use a BT?
 - When deciding *what to do next*
 - When you want *hierarchical modularity*
 - How does a BT work? (in detail)
 - How to create BTs?
 - Improvise
 - Use planning (backward chaining of sub-controllers)
 - The Big Picture
 - Genetic Algorithms
 - Control Theory (Performance Guarantees)
 - Reinforcement Learning
-



What is a Behavior Tree?

A combination of sub-controllers that:

- is Modular
 - Few dependencies between components (Important for large systems)
 - Optimally modular [1]
- has a Hierarchical structure
 - Actions exist on many levels of detail (Get tea – opening door – grasp handle)
 - Hierarchical modularity
- is equally expressive as Finite State Machines (FSMs) [2] (with internal variables)
 - choice a matter of taste (as programming languages)
- generalizes several earlier approaches [3]
 - Subsumption Architecture, Teleo-Reactive Approach, Decision Trees

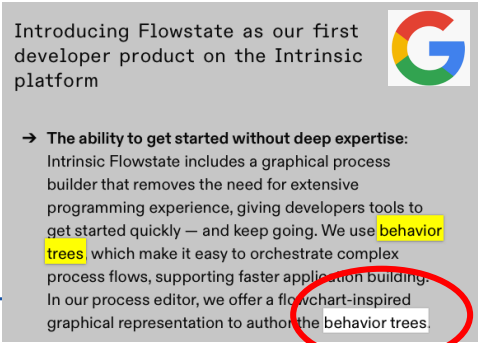
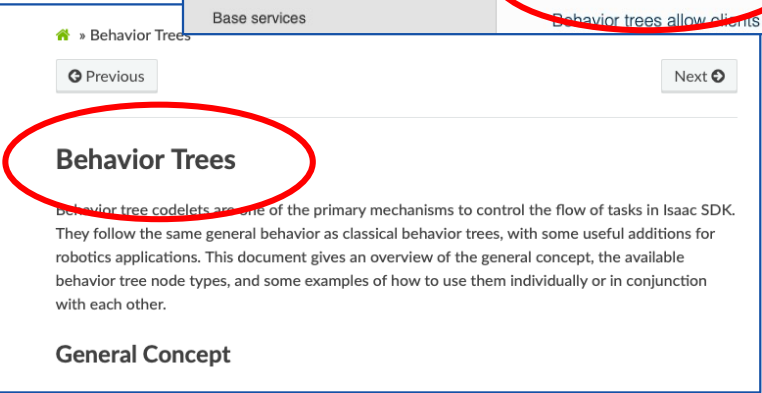
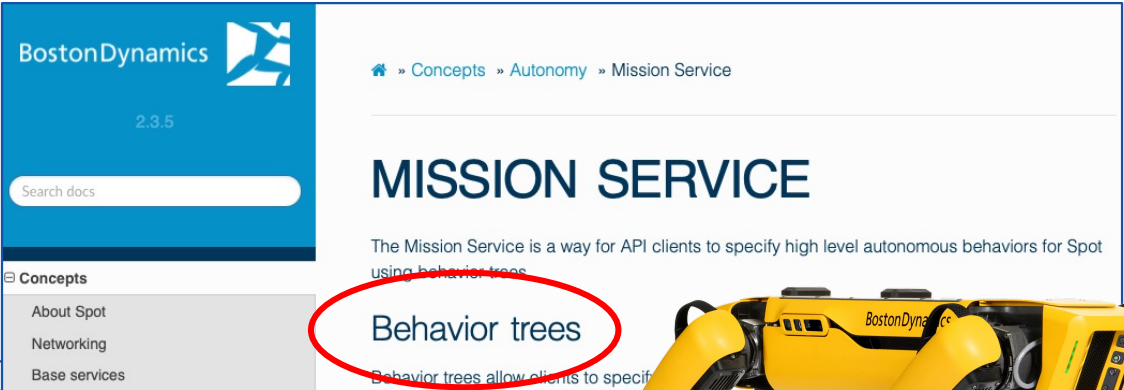
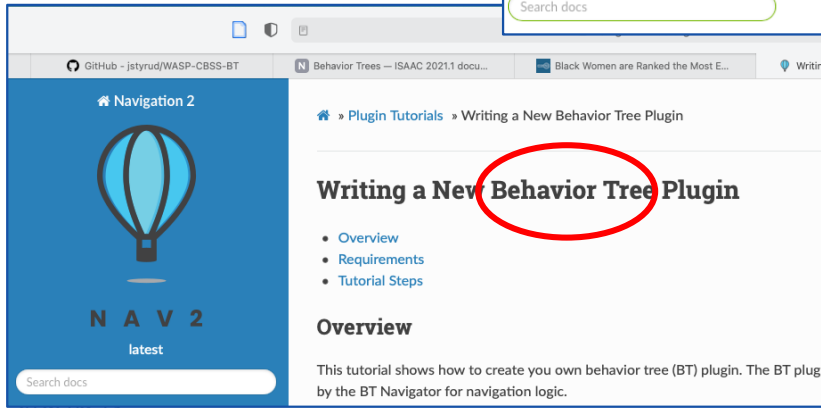




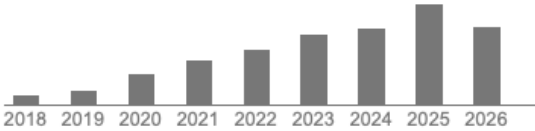
When to use Behavior trees

Invented by computer game programmers...

...refined by robotics researchers



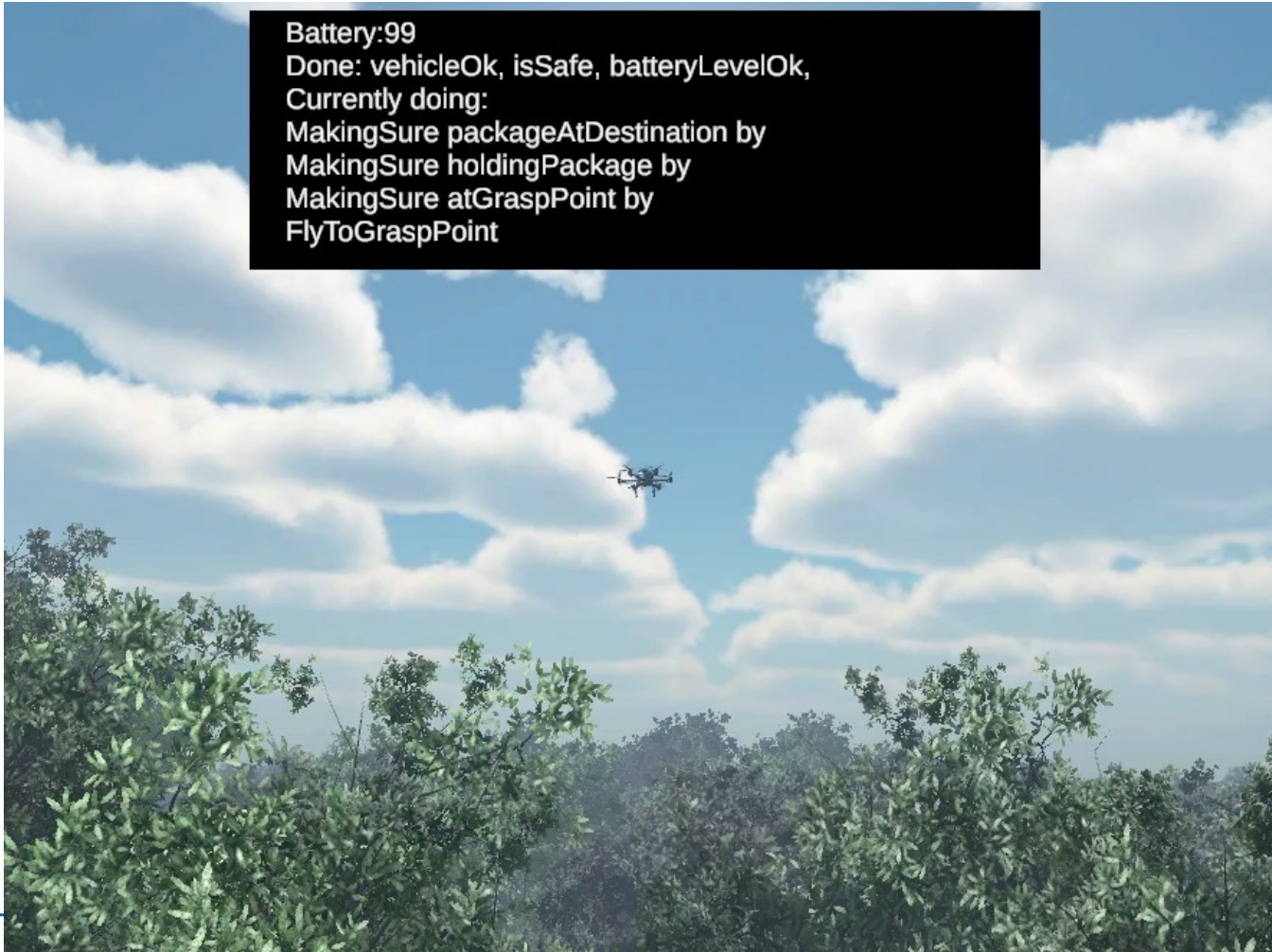
Cited by 1257





A Drone Pick-and-Place Mission

Battery:99
Done: vehicleOk, isSafe, batteryLevelOk,
Currently doing:
MakingSure packageAtDestination by
MakingSure holdingPackage by
MakingSure atGraspPoint by
FlyToGraspPoint



Tag: Untagged Layer: Default

Transform

Panda Behaviour (Script)

Tick On: Manual Repeat Root

Status:Running

Count: 1

BT Script 0 DroneTot.BT

- 1 tree "Root"
- 2 sequence
- 3 fallback // "MakeSureVehicleOk"
- 4 VehicleOk
- 5 ControlReAllocation
- 6 EmergencyLanding
- 7 EmergencyCrash
- 8 fallback // "MakeSureIsSafe"
- 9 IsSafe
- 10 AvoidCollisionAndNoFlyZones
- 11 fallback // "MakeSureBatteryOk"
- 12 BatteryLevelOk
- 13 sequence
- 14 tree "MakeSureDockedWithCharger"
- 15 Recharge
- 16 fallback // "MakeSurePackageAtDestination"
- 17 PackageAtDestination
- 18 sequence
- 19 fallback // "MakeSureHoldingPackage"
- 20 HoldingPackage
- 21 sequence
- 22 fallback // "MakeSureAtGraspPoint"
- 23 AtGraspPoint
- 24 FlyToGraspPoint
- 25 GraspPackage
- 26 fallback // "MakeSureAtDropPoint"
- 27 AtDropPoint
- 28 FlyToDropPoint
- 29 DropPackage
- 30 tree "MakeSureDockedWithCharger"

Note: Low
fidelity
(point mass)
vehicle
models



Reacting to disturbances

Battery:98
Done: vehicleOk, isSafe, batteryLevelOk,
Currently doing:
MakingSure packageAtDestination by
MakingSure holdingPackage by
MakingSure atGraspPoint by
FlyToGraspPoint

Tag Untagged Layer Default

Transform

Panda Behaviour (Script)

Tick On: Manual Repeat Root

Status:Running

Count 1

BT Script 0 DroneTot.BT

- 1 tree "Root"
- 2 sequence
- 3 fallback // "MakeSureVehicleOk"
- 4 VehicleOk
- 5 ControlReAllocation
- 6 EmergencyLanding
- 7 EmergencyCrash
- 8 fallback // "MakeSureIsSafe"
- 9 IsSafe
- 10 AvoidCollisionAndNoFlyZones
- 11 fallback // "MakeSureBatteryOk"
- 12 BatteryLevelOk
- 13 sequence
- 14 tree "MakeSureDockedWithCharger"
- 15 Recharge
- 16 fallback // "MakeSurePackageAtDestination"
- 17 PackageAtDestination
- 18 sequence
- 19 fallback // "MakeSureHoldingPackage"
- 20 HoldingPackage
- 21 sequence
- 22 fallback // "MakeSureAtGraspPoint"
- 23 AtGraspPoint
- 24 FlyToGraspPoint
- 25 GraspPackage
- 26 fallback // "MakeSureAtDropPoint"
- 27 AtDropPoint
- 28 FlyToDropPoint
- 29 DropPackage
- 30 tree "MakeSureDockedWithCharger"



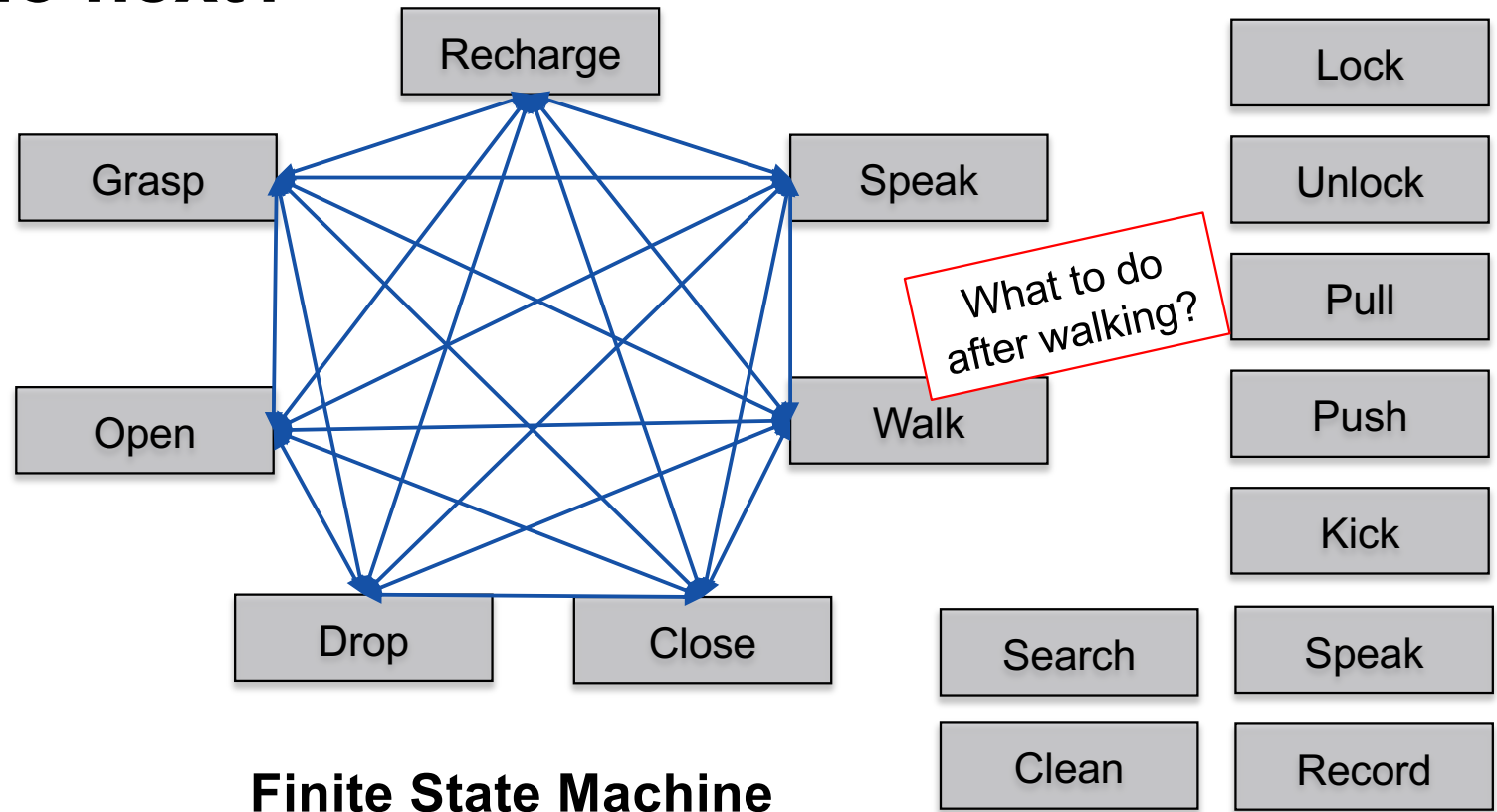
What to do next?

(most autonomous systems need to answer this question)



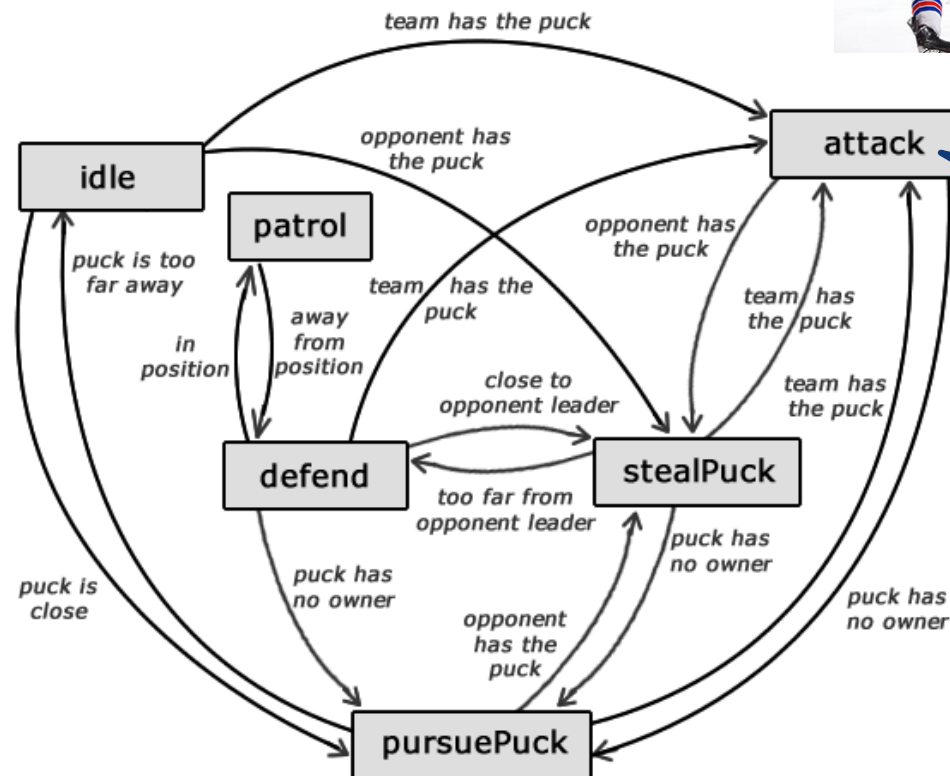
Grasp	Recharge
Drop	Lock
Walk	Unlock
Open	Pull
Close	Push
Search	Kick
Clean	Speak
Listen	Idle
Run	Throw

What to do next?



Each action needs to know “What to do next”...

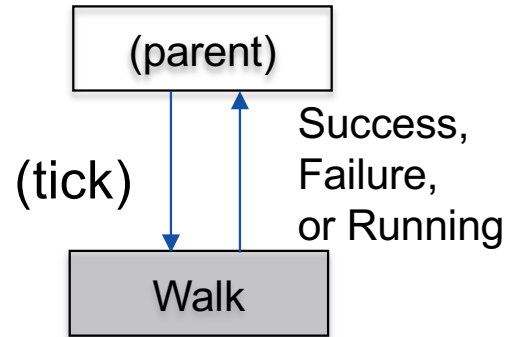
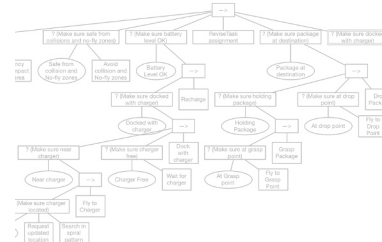
Can you spot the Bug?



Can be
expanded
...



What to do next?



Behavior Tree

Each action needs to know
“Did I Succeed or Fail?”

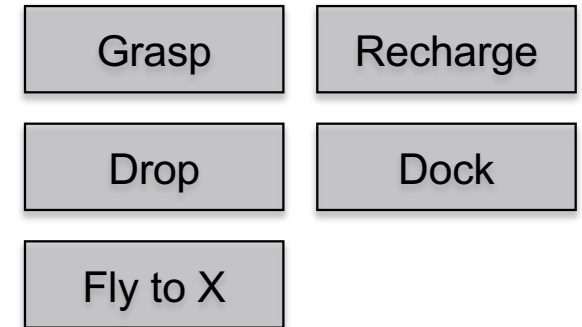
Ancestors decide “What to do next?”

Grasp	Recharge
Drop	Lock
Walk	Unlock
Open	Pull
Close	Push
Search	Kick
Clean	Speak
Speak	Record
Run	Throw



What to do next?

(Global Perspective)



Each Action/Controller/Policy has an Operating Region Ω_i

Recharge:

NotBatteryLevelOk AND DockedWithCharger

DockWithCharger:

(NotBatteryLevelOk AND DockedWithCharger AND NearCharger) OR
(PackageAtDestination AND NearCharger)

Cannot design
this manually

(NotBatteryLevelOk AND DockedWithCharger AND NotNearCharger) OR
(PackageAtDestination AND NotNearCharger)

DropPackage:

BatteryLevelOk AND Package AND AtDropPoint

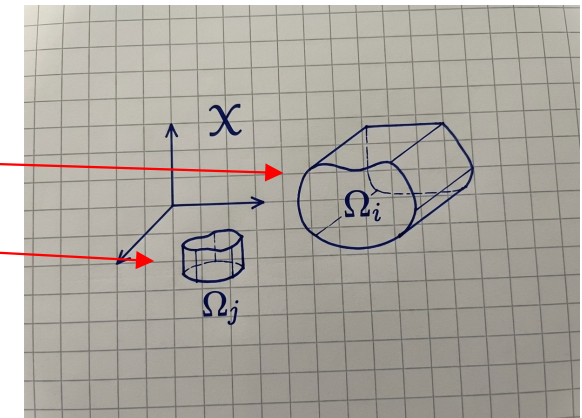
Using BTs, we
can we find it
automatically

(BatteryLevelOk AND Package AND NotAtDropPoint)

(BatteryLevelOk AND PackageAtDestination AND NotHoldingPackage AND AtGraspPoint)

(BatteryLevelOk AND NotPackageAtDestination AND NotHoldingPackage AND NotAtGraspPoint)

Every state maps
to an action!





Content

- What is a Behavior Tree?
 - A highly modular hierarchical combination of sub-controllers
 - When/Why use a BT?
 - When deciding *what to do next*
 - When you want *hierarchical modularity*
 - **How does a BT work? (in detail)**
 - How to create BTs?
 - Improvise
 - Use planning (backward chaining of sub-controllers)
 - The Big Picture
 - Genetic Algorithms
 - Control Theory (Performance Guarantees)
 - Reinforcement Learning
-



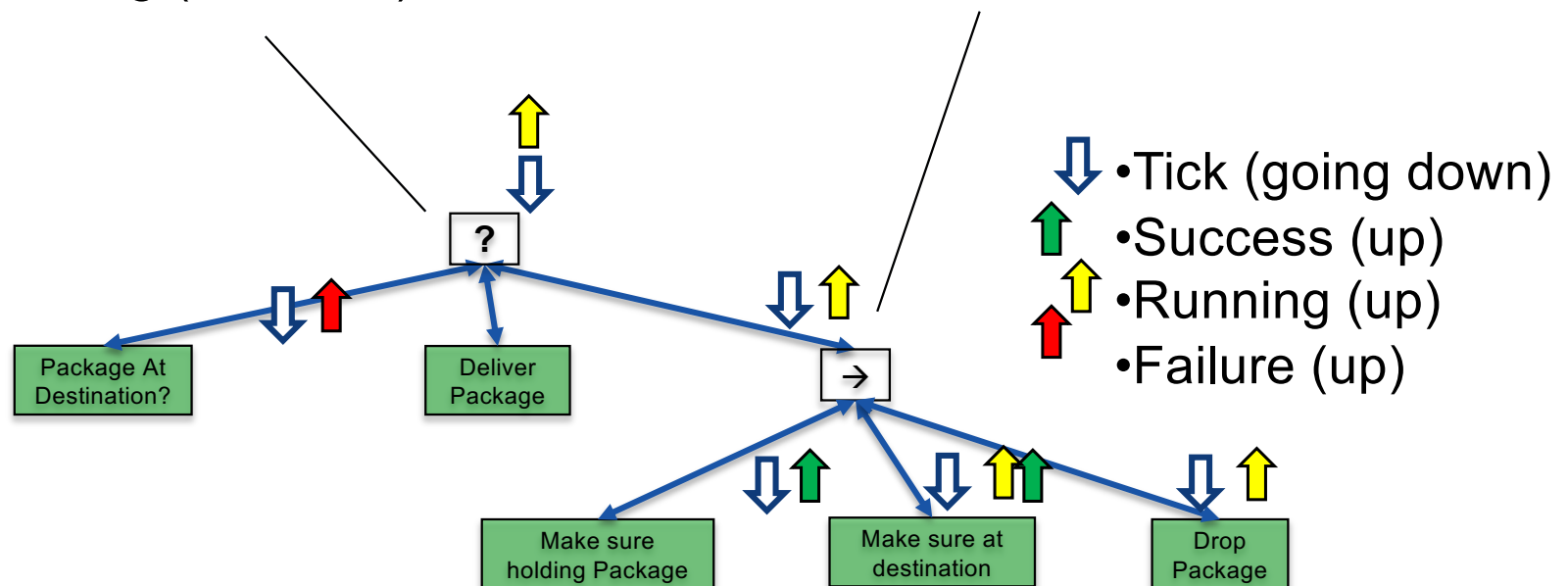
How does a Behavior Tree (BT) work?

- Fallback

- Stops at the first child that returns *success* or *running* (read “or”)

- Sequence

- Stops at the first child that returns *failure* or *running* (read “and”)



Note: We're using "Reactive" versions of all nodes

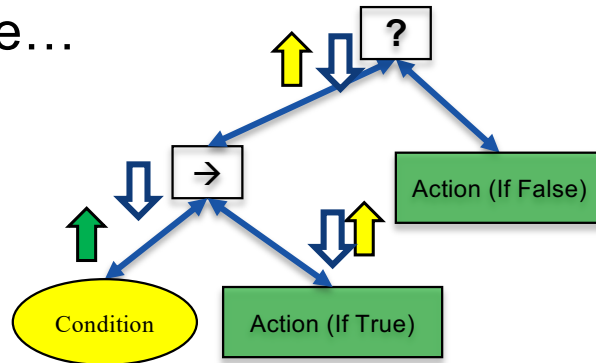


Content

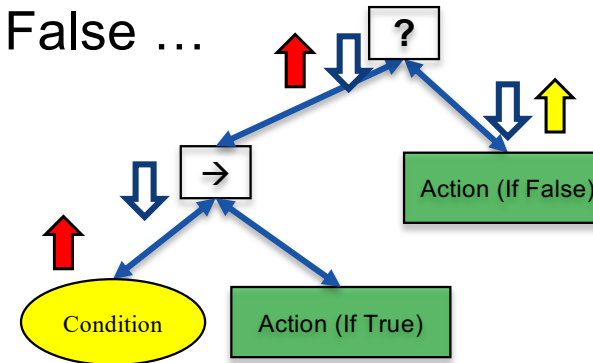
- What is a Behavior Tree?
 - A highly modular hierarchical combination of sub-controllers
 - When/Why use a BT?
 - When deciding *what to do next*
 - When you want *hierarchical modularity*
 - How does a BT work? (in detail)
 - **How to create BTs?**
 - Improvise
 - Use planning (backward chaining of sub-controllers)
 - The Big Picture
 - Genetic Algorithms
 - Control Theory (Performance Guarantees)
 - Reinforcement Learning
-

We can do nested *If-then-else* ...

- If True...



- If False ...



- We don't want *Nested If-statements*

(hard to read)

- We want
 - hierarchical structure
 - easy to read
 - clear intended progression
 - → Back-chaining...





Design BT using Planning (Backward Chaining)

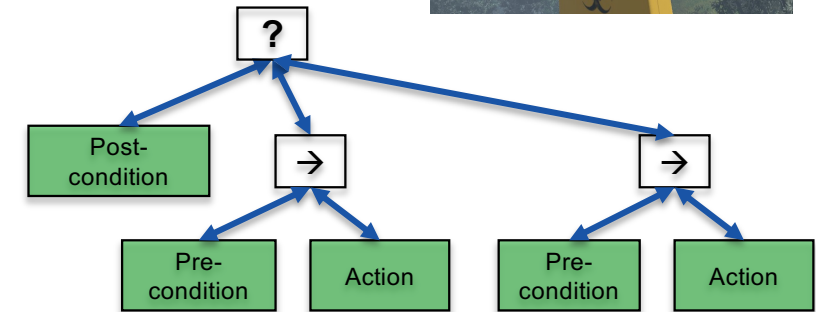
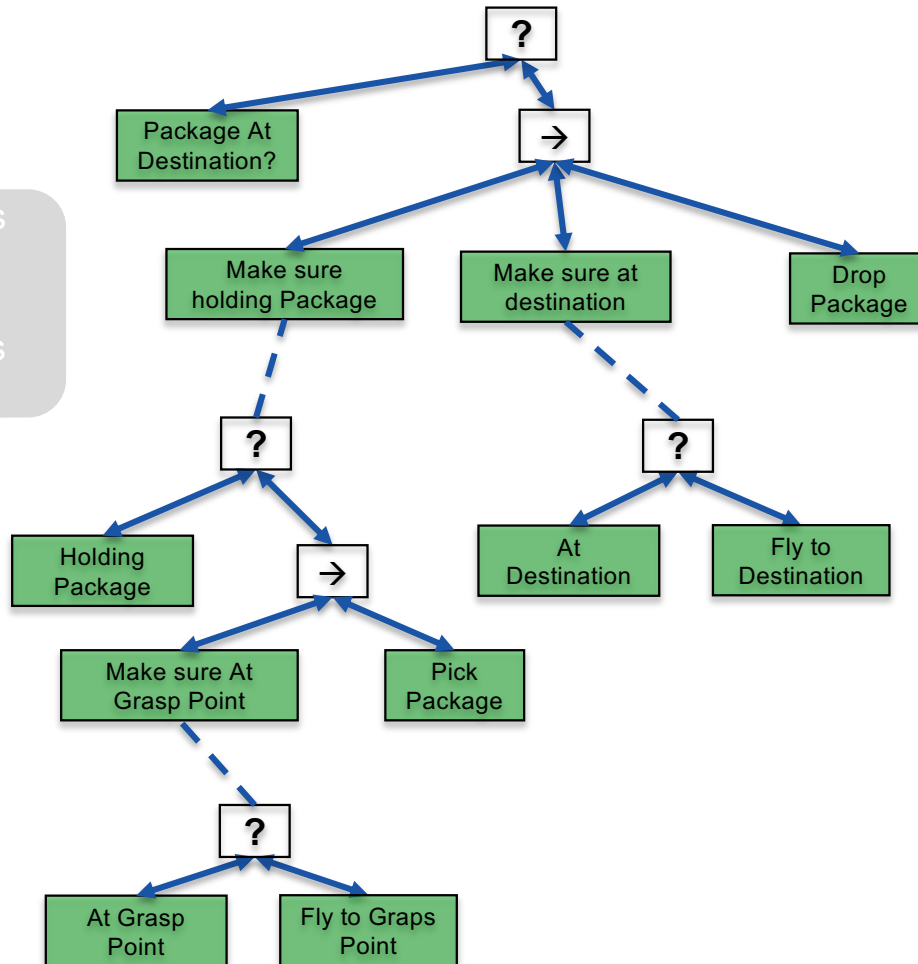
- Backward Chaining
 - Solving an AI Planning Problem by working **backwards** from the goal
- Example:
 - Goal: *Leave the room*
 - To leave I need to **pass through the door**
 - To pass the door I need to **open the door**
 - To open the door I need to **grasp the handle**
 - To grasp the handle I need to **extend my arm**
 - **Plan:**
 - > *Extend arm*
 - > *Grasp handle*
 - > *Open door*
 - > *Pass through the door*

BTs can do this reactively...

Back-chaining a BT



Simple BTs that make sure a condition is satisfied

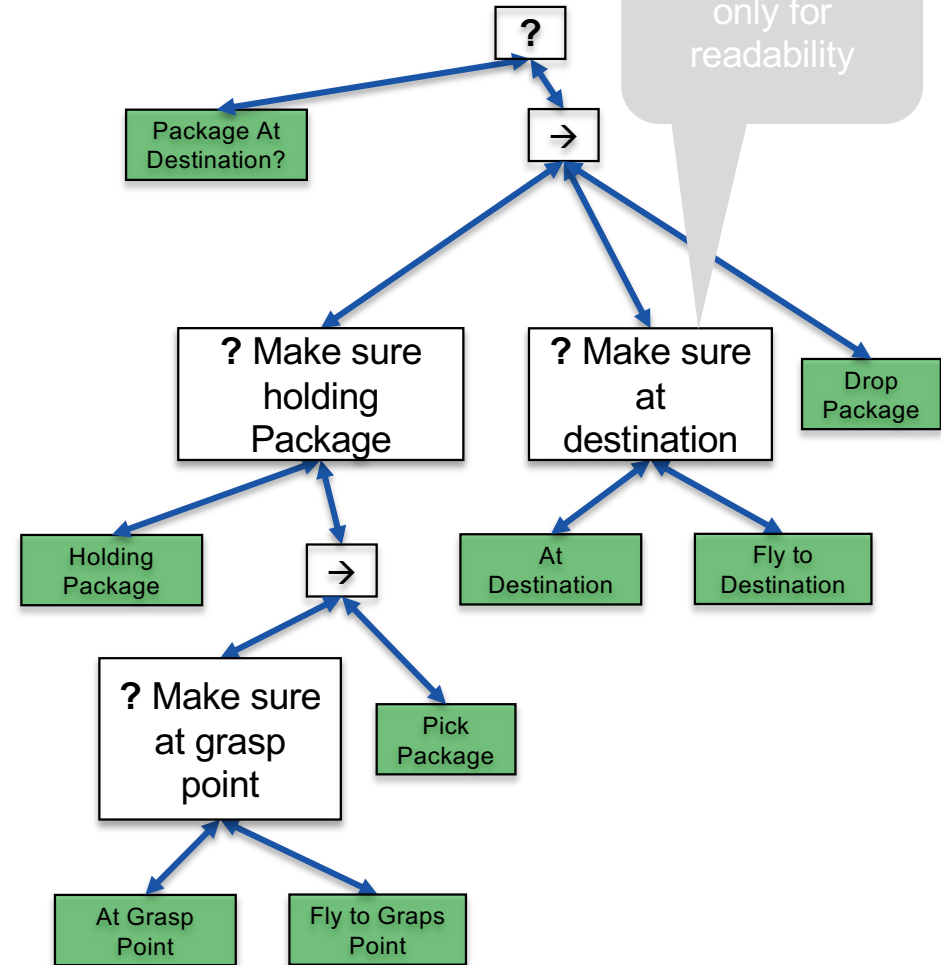
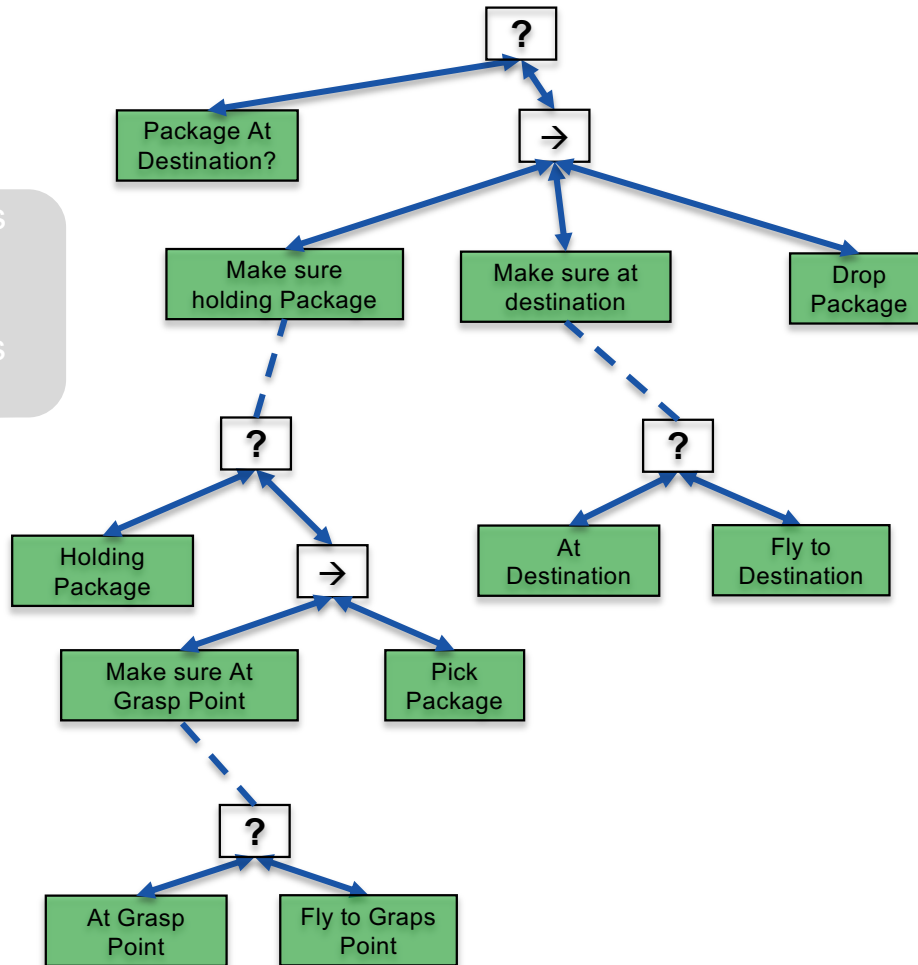


Sometimes called:

- Postcondition-Precondition-Action BT (PPA-BT)
- Make-sure BT

Back-chaining a BT

Simple BTs that make sure a condition is satisfied



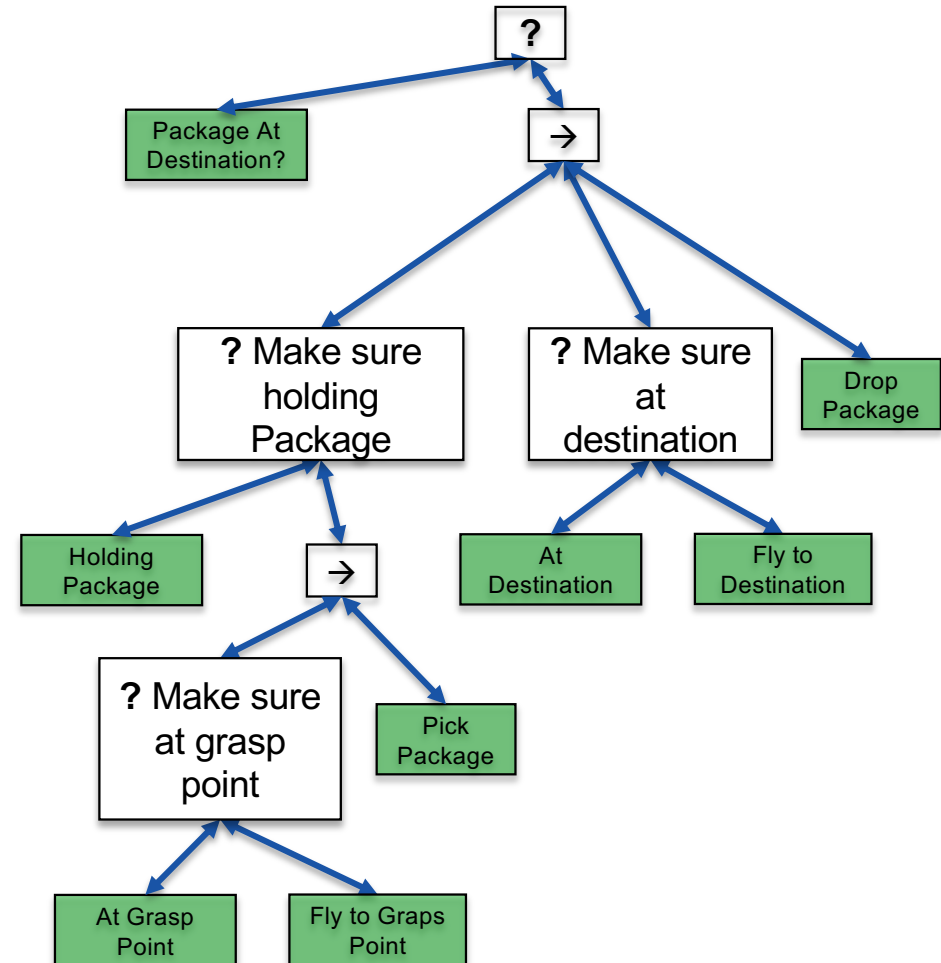
Any Additional Goals?

Goals

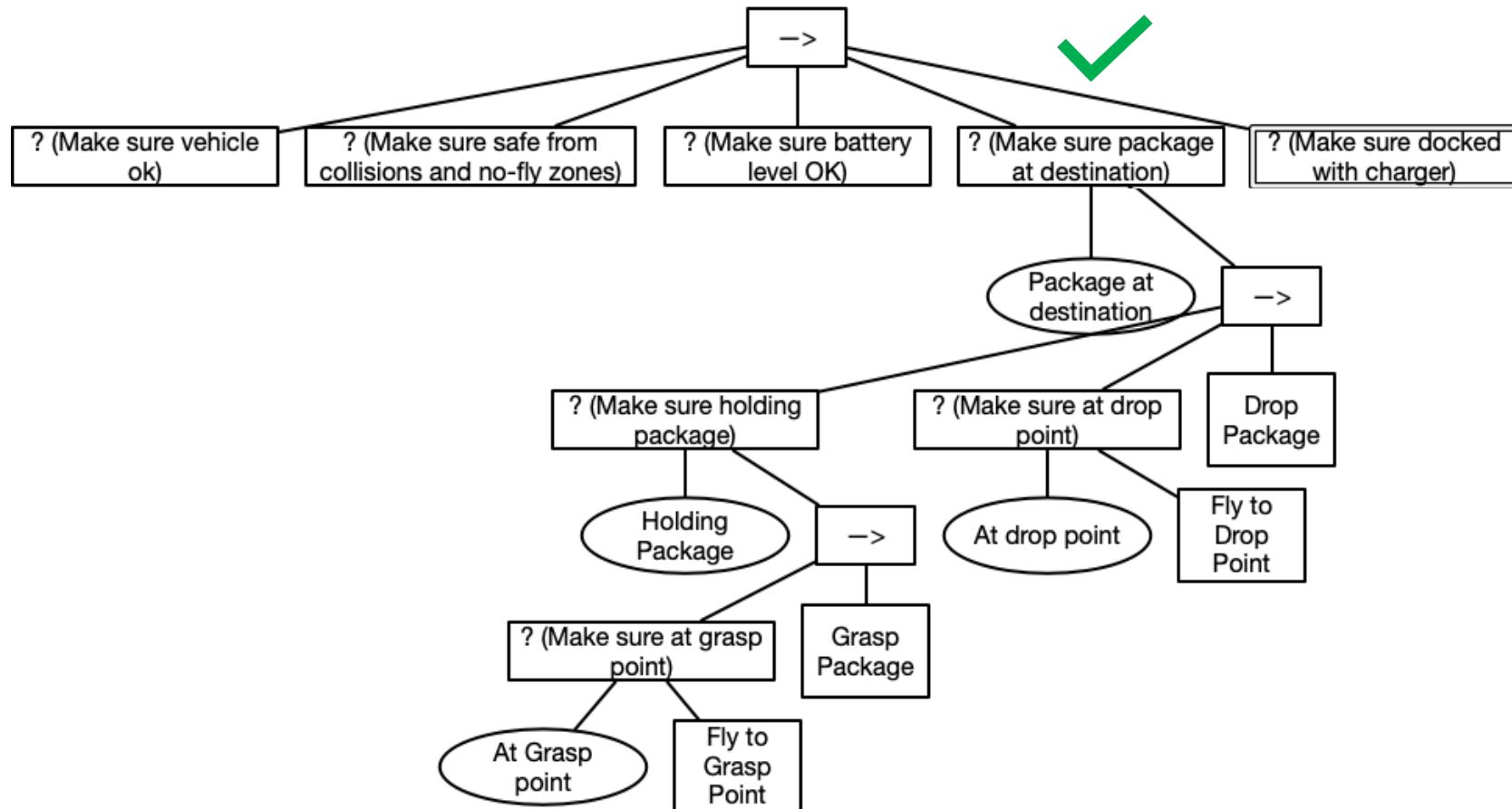
- Avoid
 - Collisions
 - No-fly zones
 - Empty battery
- Land when mission is done

Back-chaining Algorithm

- List top level goals (in Sequence)
- Do back-chaining on those

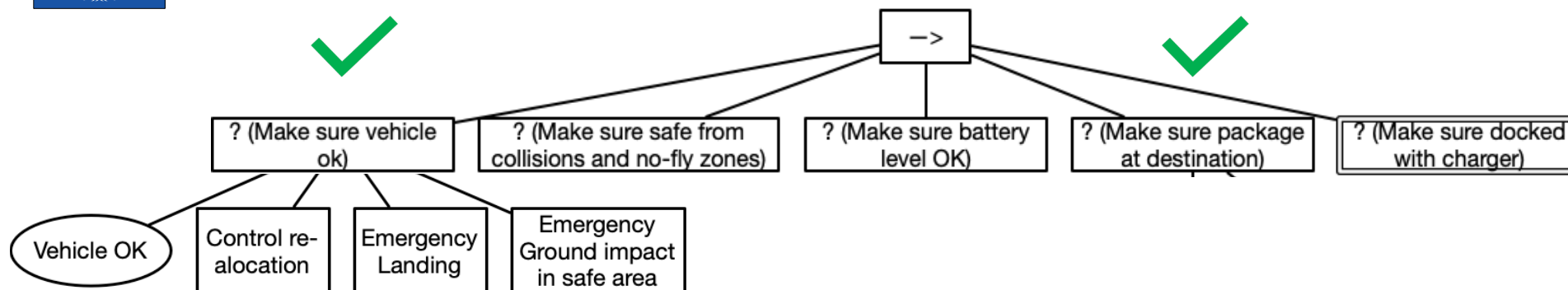


The top level goals





The top level goals



Inspired by

2019 International Conference on Unmanned Aircraft Systems (ICUAS)
Atlanta, GA, USA, June 11-14, 2019

Safe decision making for risk mitigation of UAS

Lina Castano, and Huan Xu, *Member, IEEE*

2016 American Control Conference (ACC)
Boston Marriott Copley Place
July 6-8, 2016. Boston, MA, USA

Adaptive Fault Tolerant Control Allocation for a Hexacopter System*

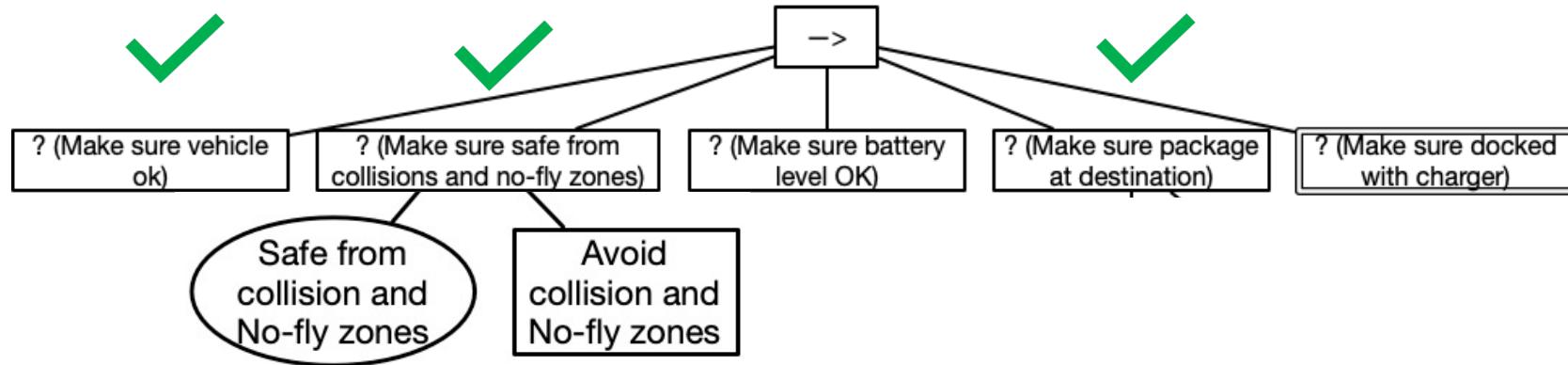
Guillermo P. Falconi¹ and Florian Holzapfel¹

Abstract—In this paper an adaptive fault tolerant control allocation for a linear static actuator model is presented. The method proposed here has the advantage of not relying on

Fault tolerance is closely related to over-actuated systems, where the number of total forces and moments available to control the system (virtual inputs) are less than the number of

of a theoretical for small UAS operations to be conducted routinely within
manned Aerial the visual line of sight (VLOS). A comprehensive set of
framework uses additional regulations for sUAS were provided by the FAA's

The top level goals



2008 IEEE International Conference on
Robotics and Automation
Pasadena, CA, USA, May 19-23, 2008

Reciprocal Velocity Obstacles for Real-Time Multi-Agent Navigation

Jur van den Berg

Ming Lin

Dinesh Manocha

Abstract—In this paper, we propose a new concept — the “Reciprocal Velocity Obstacle”— for real-time multi-agent navigation. We consider the case in which each agent navigates independently without explicit communication with other agents. Our formulation is an extension of the Velocity Obstacle concept [3], which was introduced for navigation among (passively) moving obstacles. Our approach takes into account the reactive behavior of the other agents by implicitly assuming that

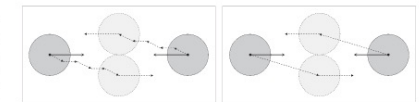
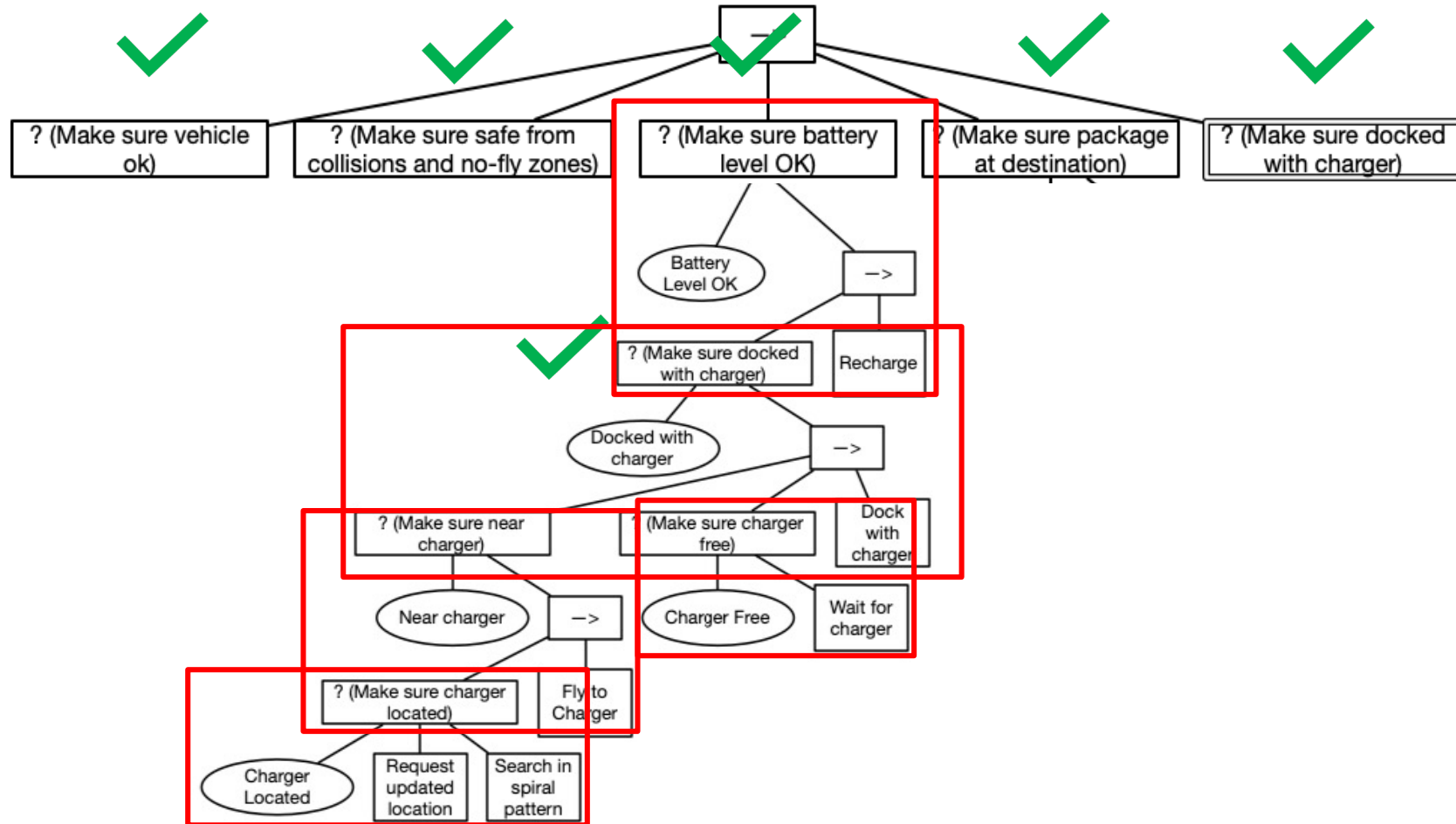


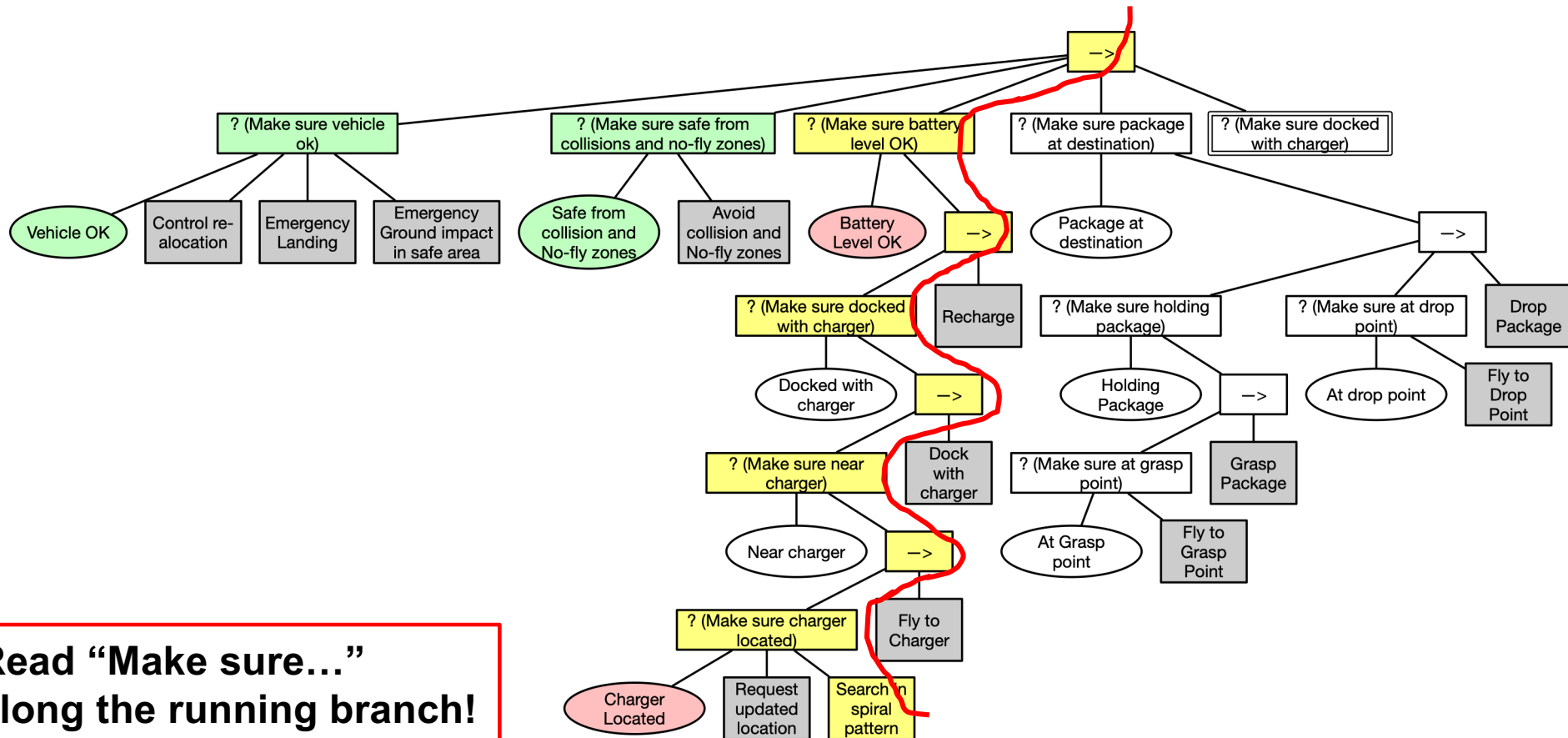
Fig. 1. The paths followed by two agents that have opposite preferred

The top level goals





What about readability?



Battery:95
Done: vehicleOk, isSafe, batteryLevelOk,
Currently doing:
MakingSure packageAtDestination by
MakingSure holdingPackage by
MakingSure atGraspPoint by
FlyToGraspPoint



Read "Make sure..."
along the running branch
(blue)!

Tag: Untagged Layer: Default

Transform

Panda Behaviour (Script)

Tick On: Manual Repeat Root

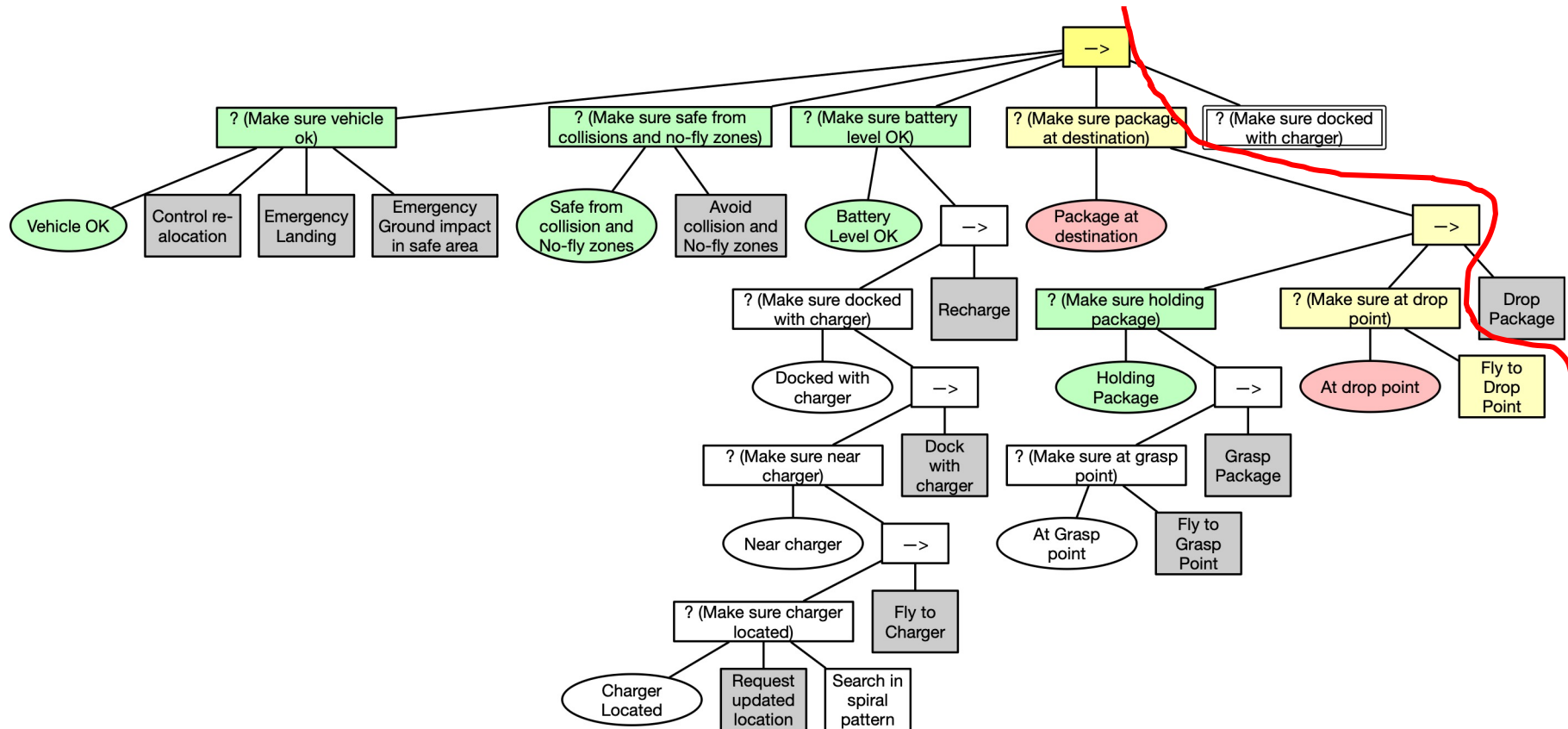
Status:Running

Count: 1

BT Script 0: DroneTot.BT

- 1 tree "Root"
- 2 sequence
- 3 fallback // "MakeSureVehicleOk"
- 4 VehicleOk
- 5 ControlReAllocation
- 6 EmergencyLanding
- 7 EmergencyCrash
- 8 fallback // "MakeSureIsSafe"
- 9 IsSafe
- 10 AvoidCollisionAndNoFlyZones
- 11 fallback // "MakeSureBatteryOk"
- 12 BatteryLevelOk
- 13 sequence
- 14 tree "MakeSureDockedWithCharger"
- 15 Recharge
- 16 fallback // "MakeSurePackageAtDestination"
- 17 PackageAtDestination
- 18 sequence
- 19 fallback // "MakeSureHoldingPackage"
- 20 HoldingPackage
- 21 sequence
- 22 fallback // "MakeSureAtGraspPoint"
- 23 AtGraspPoint
- 24 FlyToGraspPoint
- 25 GraspPackage
- 26 fallback // "MakeSureAtDropPoint"
- 27 AtDropPoint
- 28 FlyToDropPoint
- 29 DropPackage
- 30 tree "MakeSureDockedWithCharger"
- 31

What about readability?



Battery:79
Done: vehicleOk, isSafe, batteryLevelOk,
holdingPackage,
Currently doing:
MakingSure packageAtDestination by
MakingSure atDropPoint by
FlyToDropPoint



Tag: Untagged Layer: Default

Transform

Panda Behaviour (Script)

Tick On: Manual Repeat Root

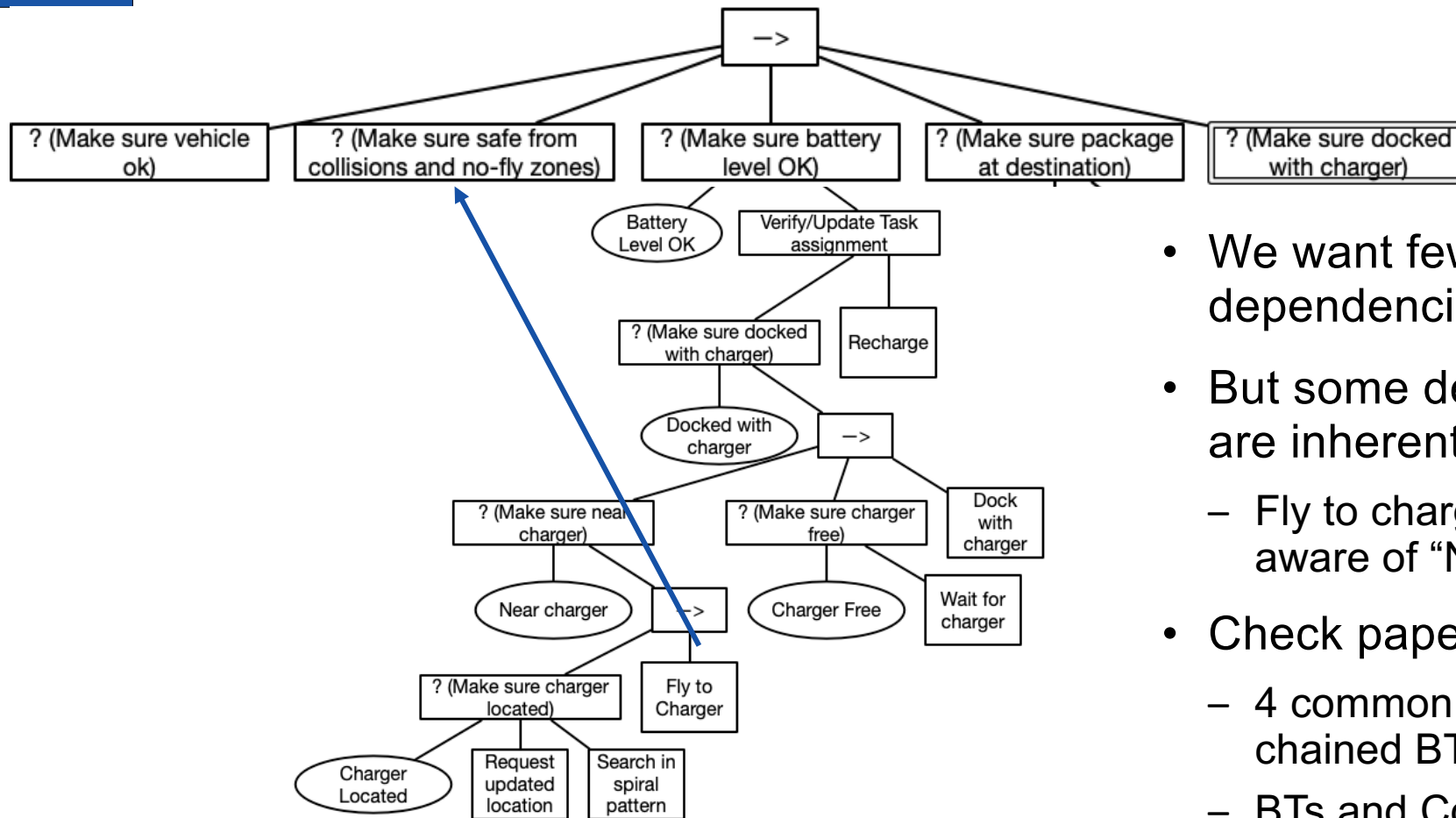
Status:Running

Count: 1

BT Script 0: DroneTot.BT

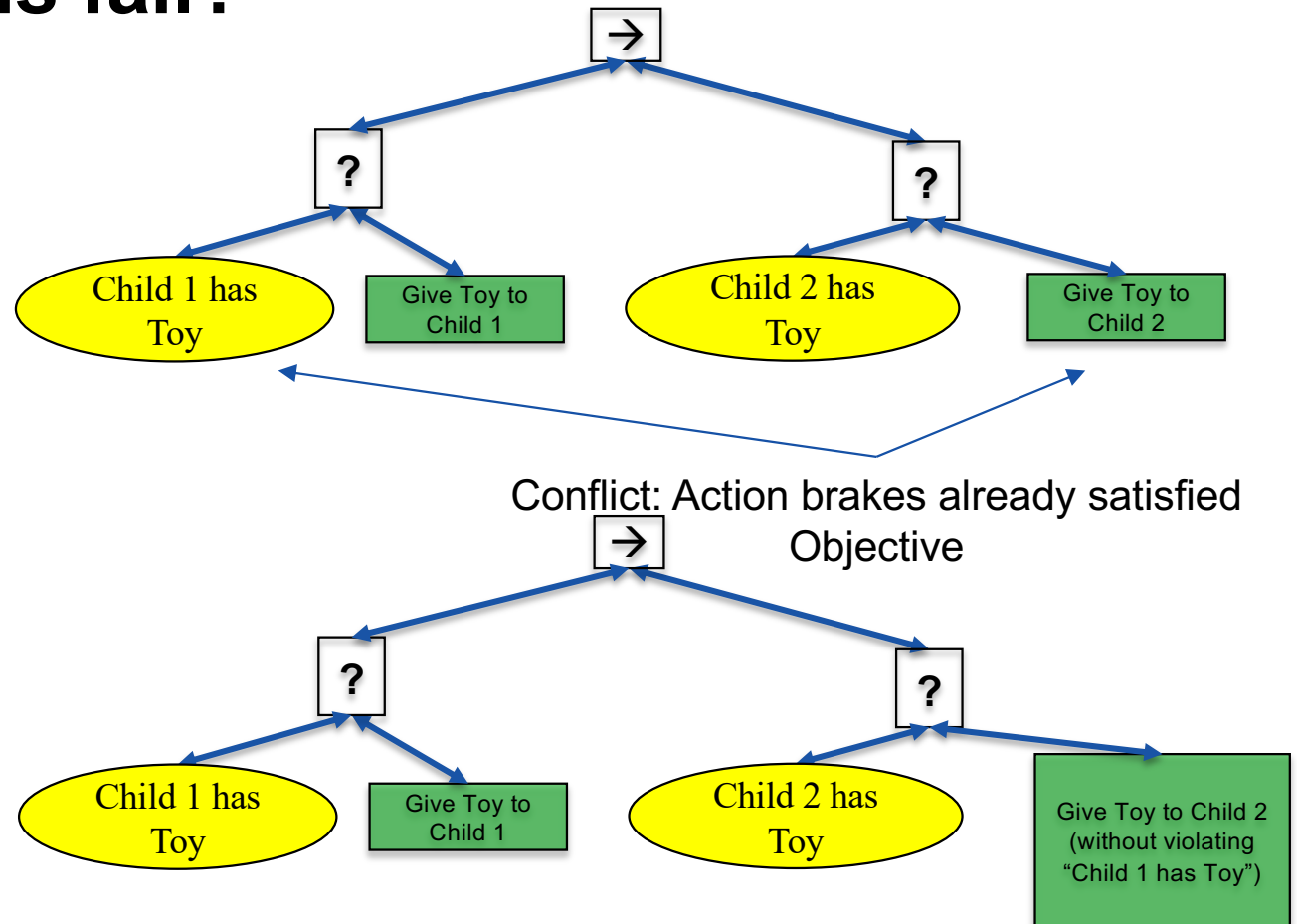
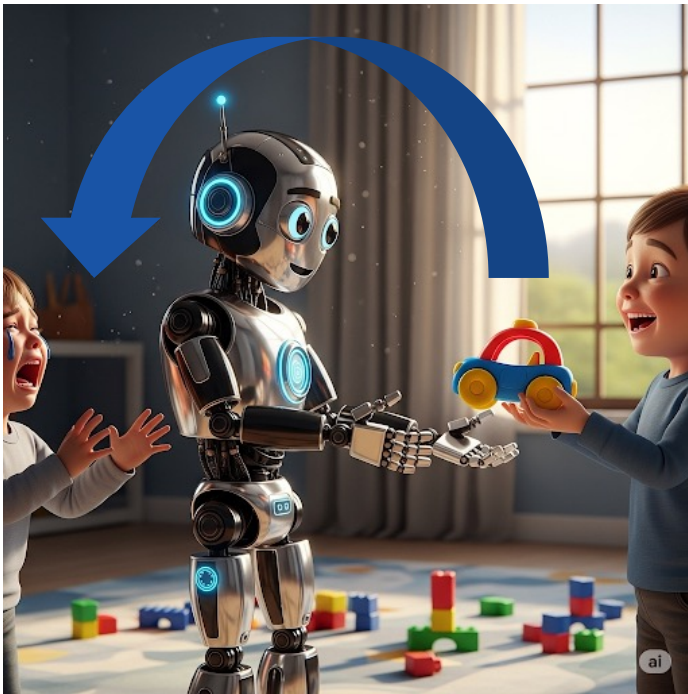
- 1 tree "Root"
- 2 sequence
 - 3 fallback // "MakeSureVehicleOk"
 - 4 VehicleOk
 - 5 ControlReAllocation
 - 6 EmergencyLanding
 - 7 EmergencyCrash
 - 8 fallback // "MakeSureIsSafe"
 - 9 IsSafe
 - 10 AvoidCollisionAndNoFlyZones
 - 11 fallback // "MakeSureBatteryOk"
 - 12 BatteryLevelOk
 - 13 sequence
 - 14 tree "MakeSureDockedWithCharger"
 - 15 Recharge
 - 16 fallback // "MakeSurePackageAtDestination"
 - 17 PackageAtDestination
 - 18 sequence
 - 19 fallback // "MakeSureHoldingPackage"
 - 20 HoldingPackage
 - 21 sequence
 - 22 fallback // "MakeSureAtGraspPoint"
 - 23 AtGraspPoint
 - 24 FlyToGraspPoint
 - 25 GraspPackage
 - 26 fallback // "MakeSureAtDropPoint"
 - 27 AtDropPoint
 - 28 FlyToDropPoint
 - 29 DropPackage
 - 30 tree "MakeSureDockedWithCharger"

Be aware of dependencies



- We want few dependencies (modularity)
- But some dependencies are inherent in the problem
 - Fly to charger needs to be aware of “No-fly zones”
- Check papers/videos on
 - 4 common bugs in back-chained BTs
 - BTs and Control Barrier Functions

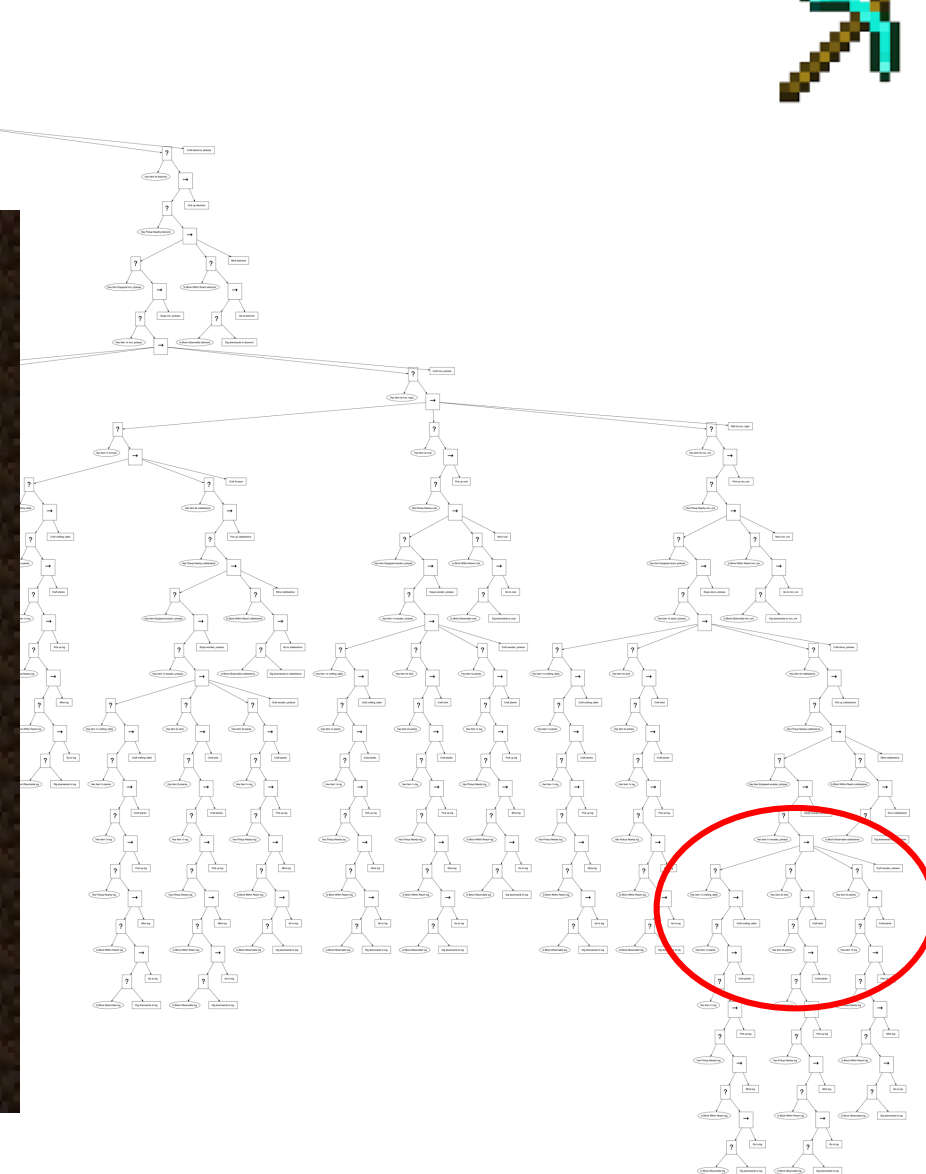
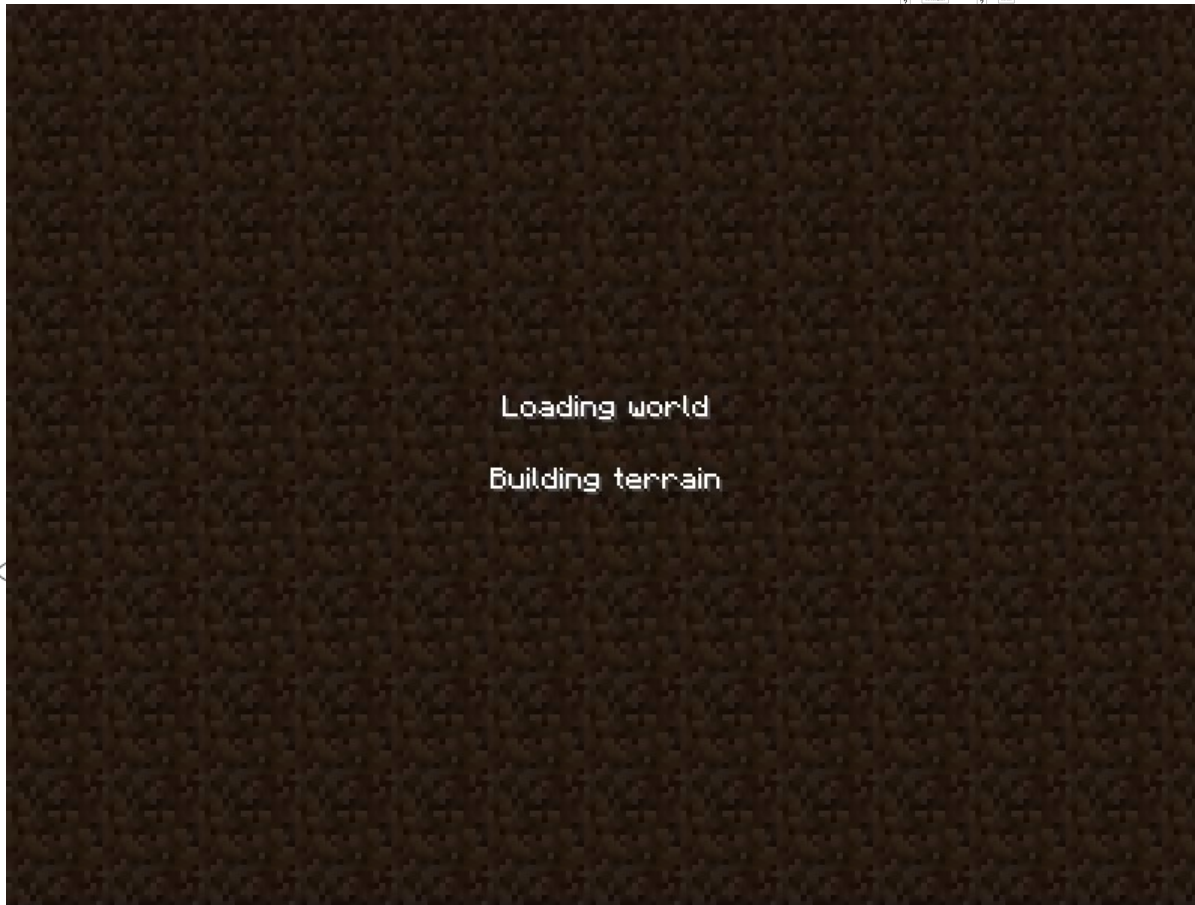
When does this fail?



Solution: Avoid braking already achieved goals (if possible)



Backward chained BT for Minecraft AI

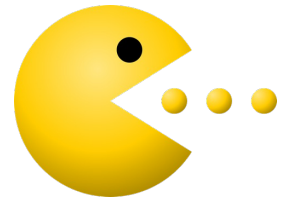


-
- ```

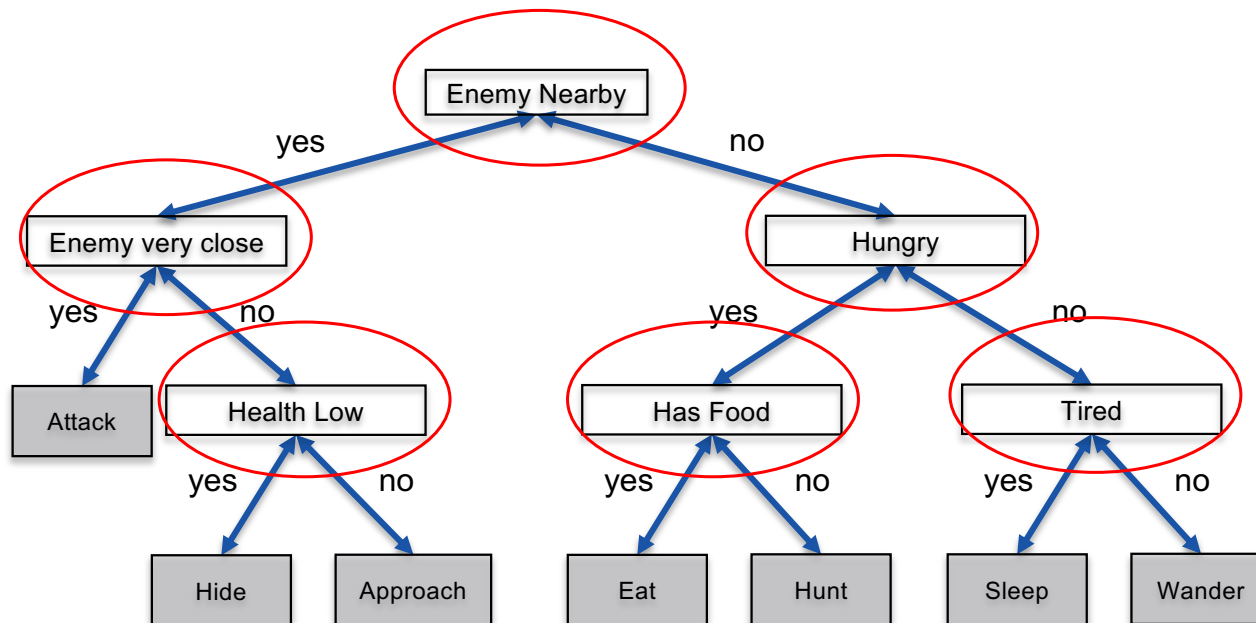
graph TD
 GC1[Ghost Close] -- yes --> GS1[Ghost Scared]
 GC1 -- no --> EP1[Eat Pills]
 GS1 --> CG1[Chase Ghost]
 GS1 --> AV1[Avoid Ghost]

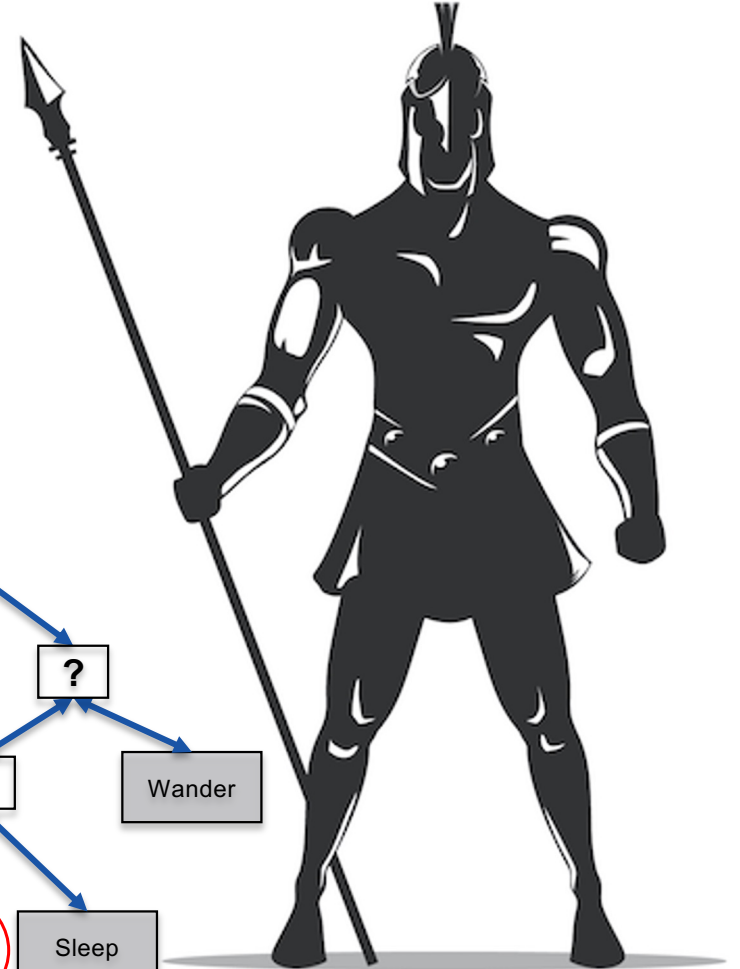
 Q1[?] --> R1[→]
 Q1 --> EP2[Eat Pills]
 R1 --> Q2[?]
 R1 --> GC2((Ghost Close))
 Q2 --> AV2[Avoid Ghost]
 Q2 --> R2[→]
 R2 --> GS2((Ghost Scared))
 R2 --> CG2[Chase Ghost]
 Q3[?] --> R3[→]
 Q3 --> CG3[Chase Ghost]

```



# Warrior DT design







# What design to use?

- If action choices can be broken down into Yes/No cases
  - Use DT design
- If you want a deliberative system that pursues sub-goals
  - Use Backward chaining designs
- Combine when needed...

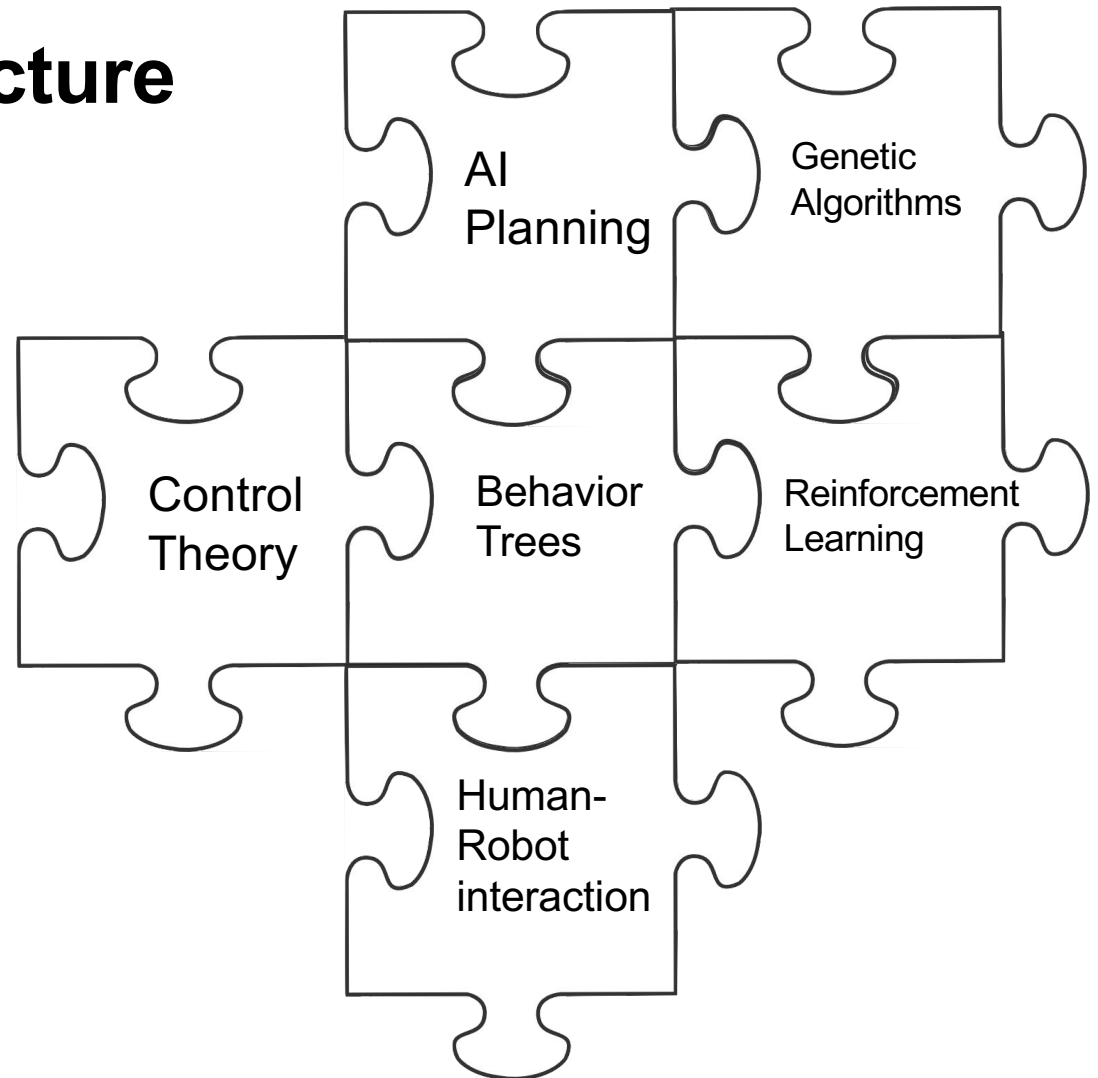


# Content

- What is a Behavior Tree?
    - A highly modular hierarchical combination of sub-controllers
  - When/Why use a BT?
    - When deciding *what to do next*
    - When you want *hierarchical modularity*
  - How does a BT work? (in detail)
  - How to create BTs?
    - Improvise
    - Use planning (backward chaining of sub-controllers)
  - **The Big Picture**
    - Genetic Algorithms
    - Control Theory (Performance Guarantees)
    - Reinforcement Learning
-



# The Big Picture

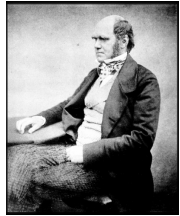
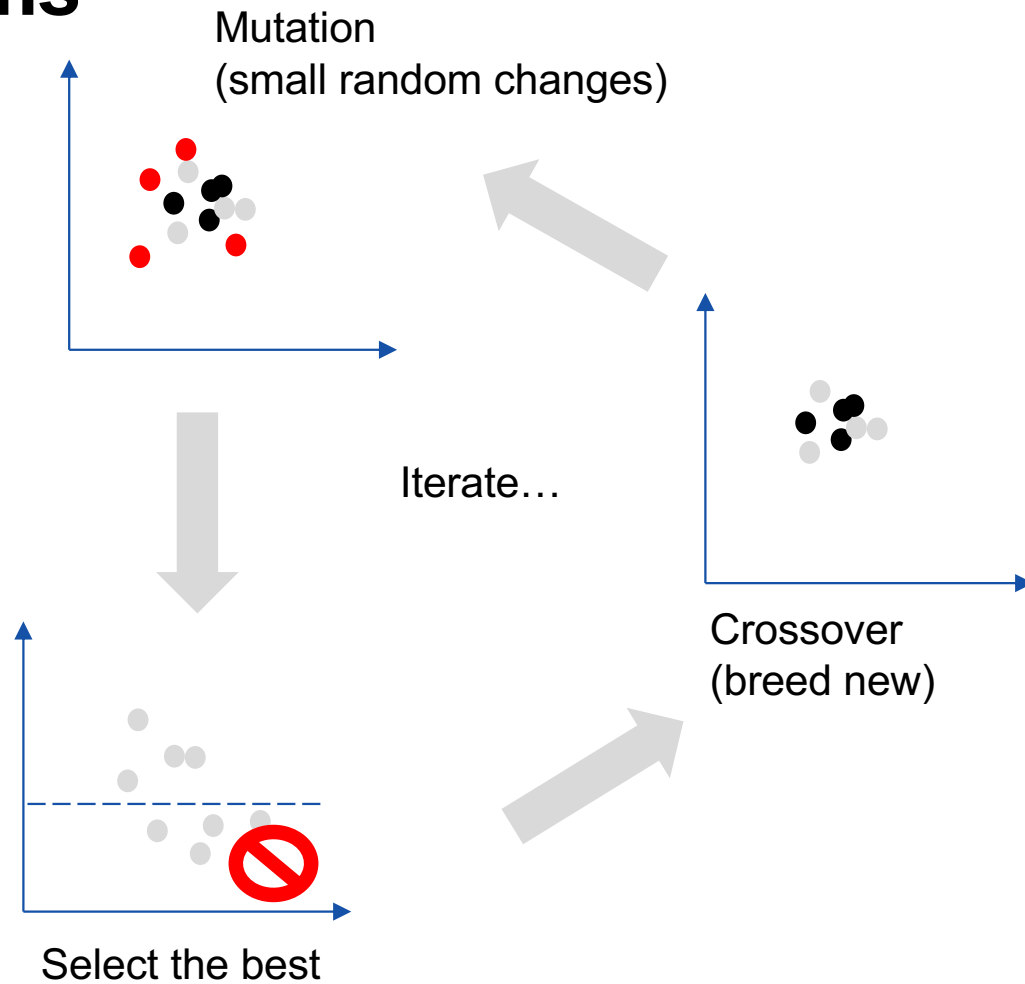
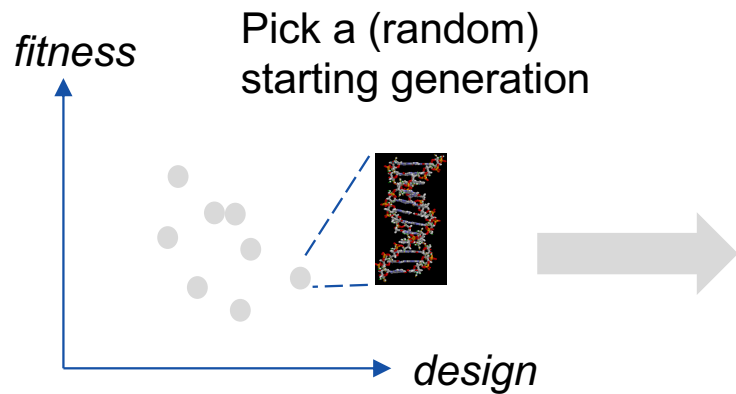




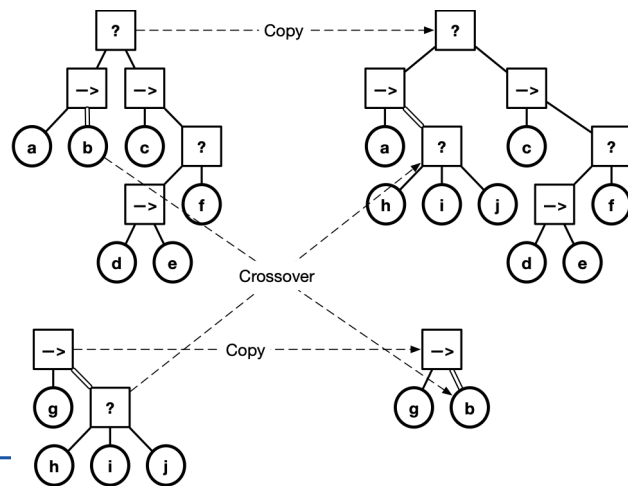
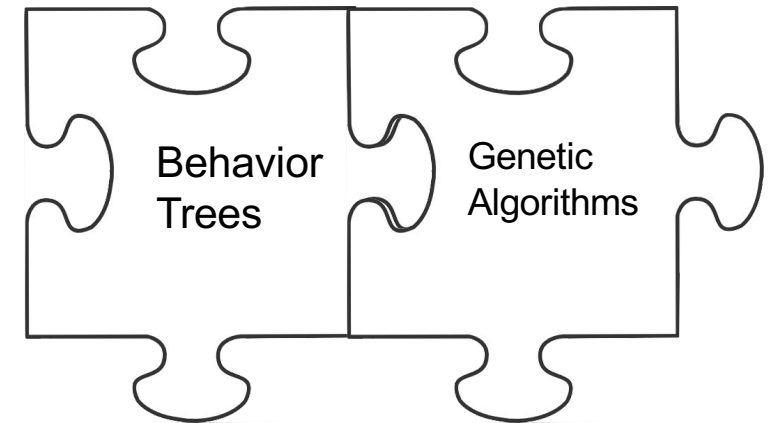
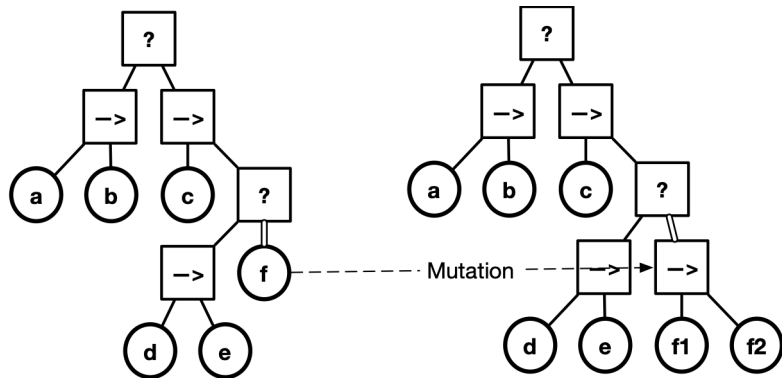
# Genetic Algorithms

Applies to

- Viruses
- Humans
- Any Optimization problem
- Behavior Trees



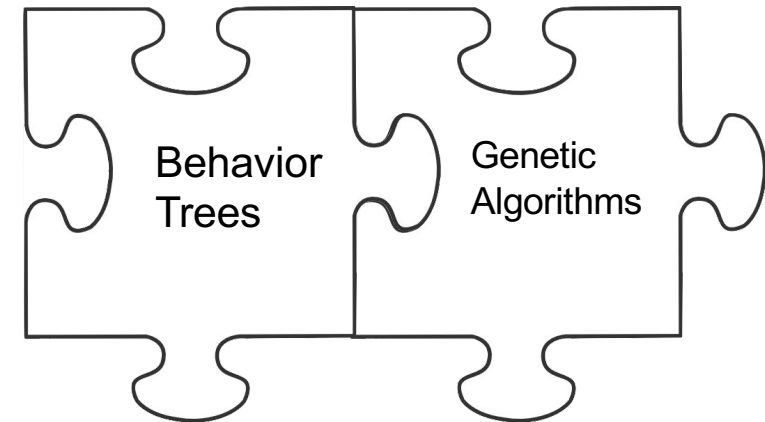
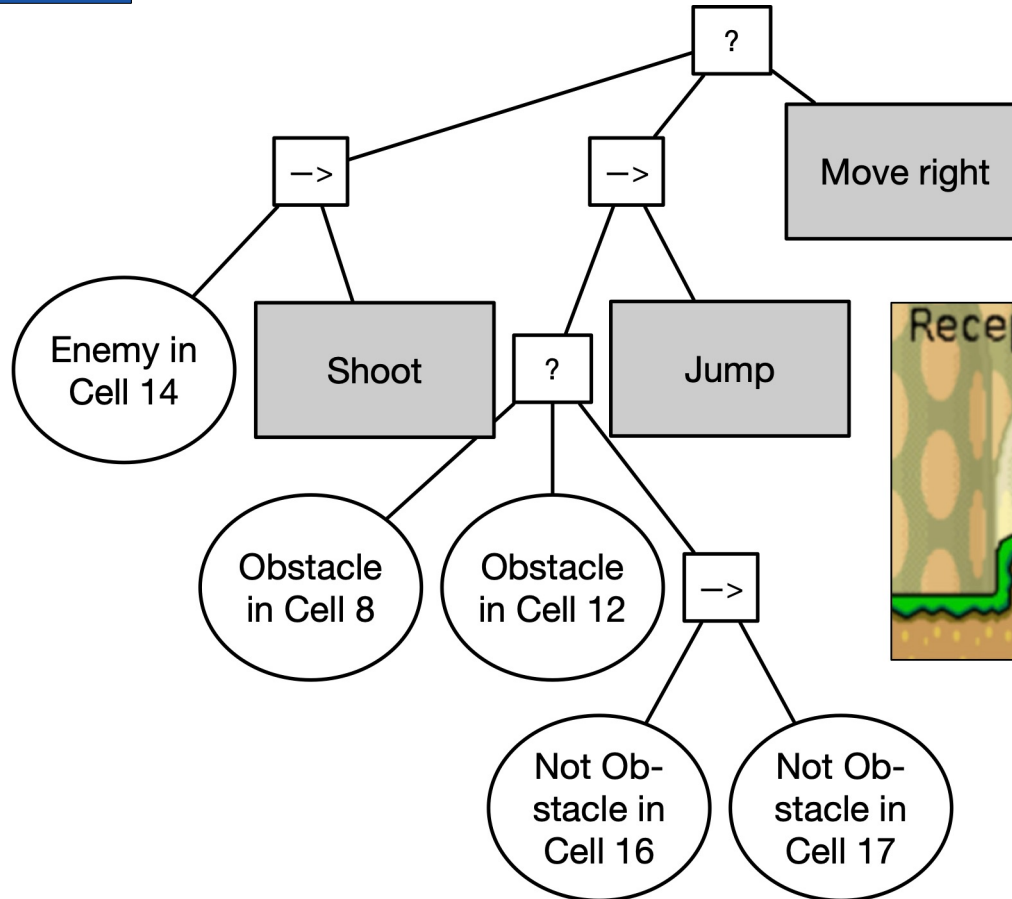
# Genetic Algorithms



- BT trivially maps to Genes
- Mutation/Crossover easy

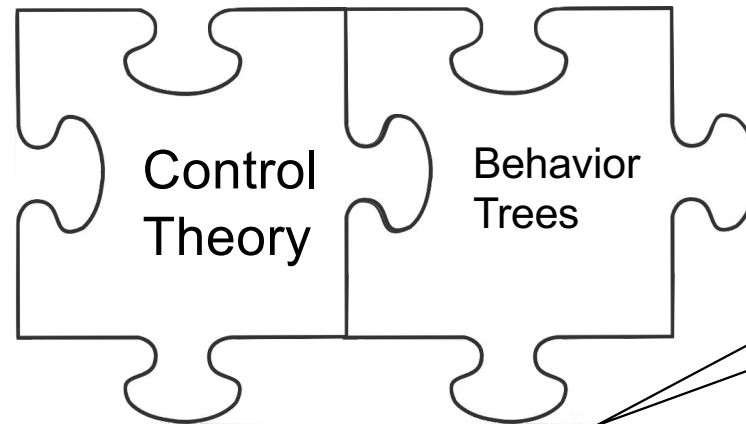


# Genetic Algorithm for Mario AI BT

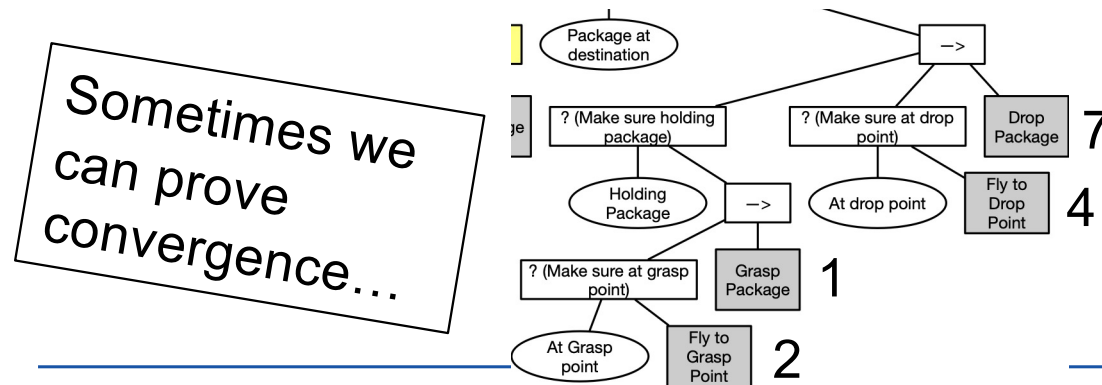
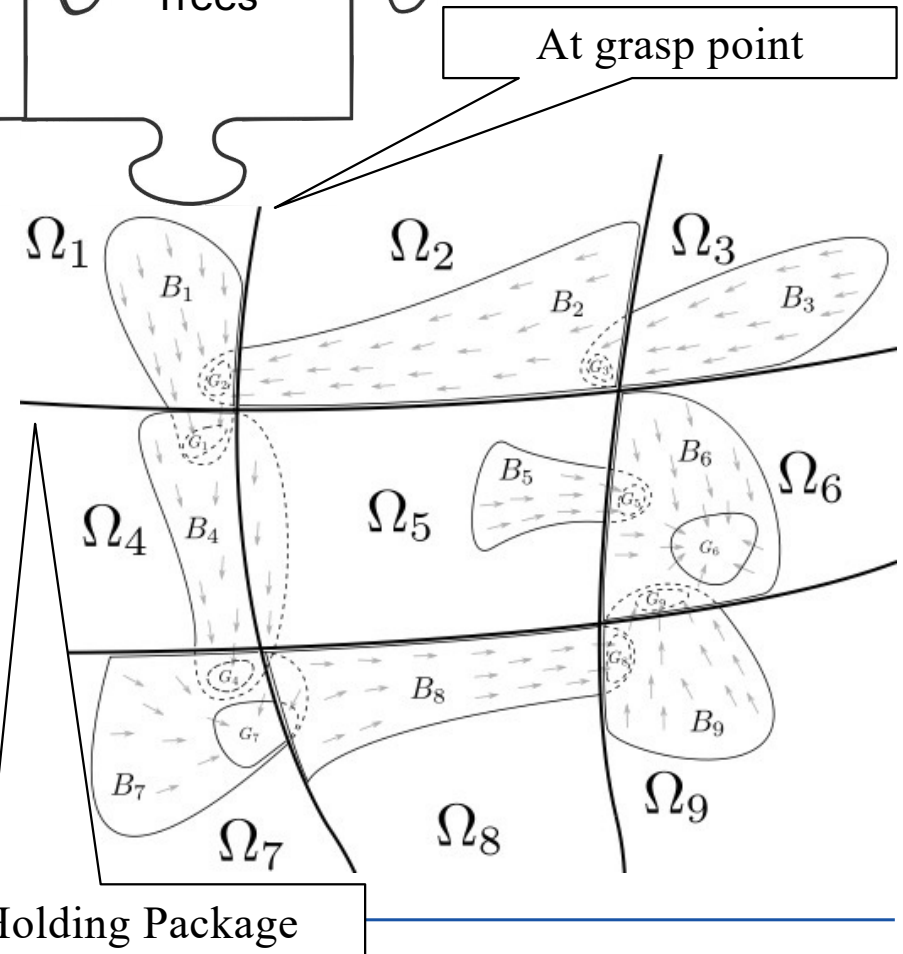




# The Big Picture

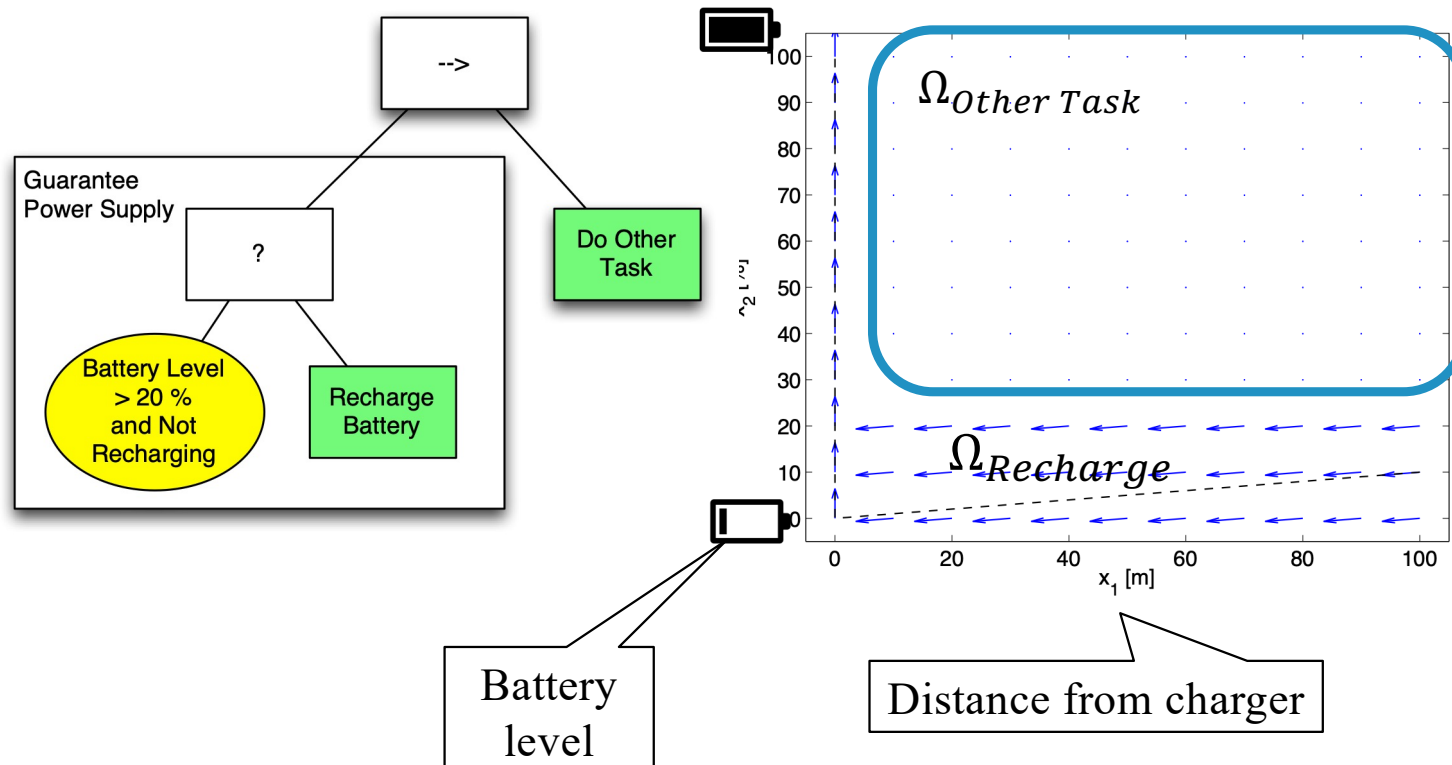


- Let actions have indexes  $i$
- Let  $\Omega_i$  be states where  $i$  executes
- Let  $B_i$  be domain of attraction

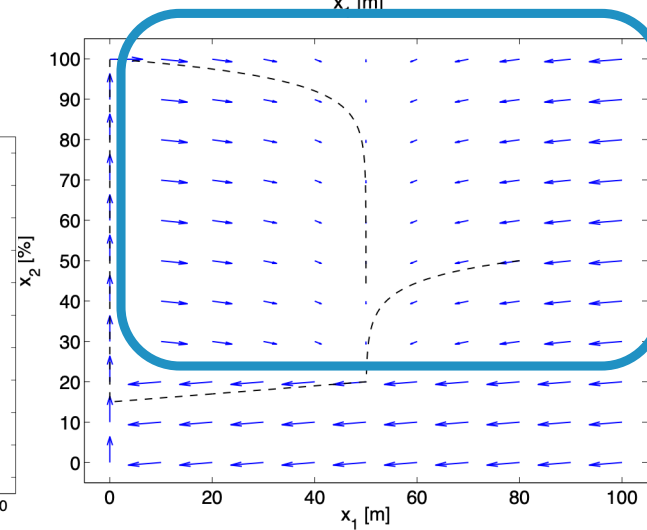
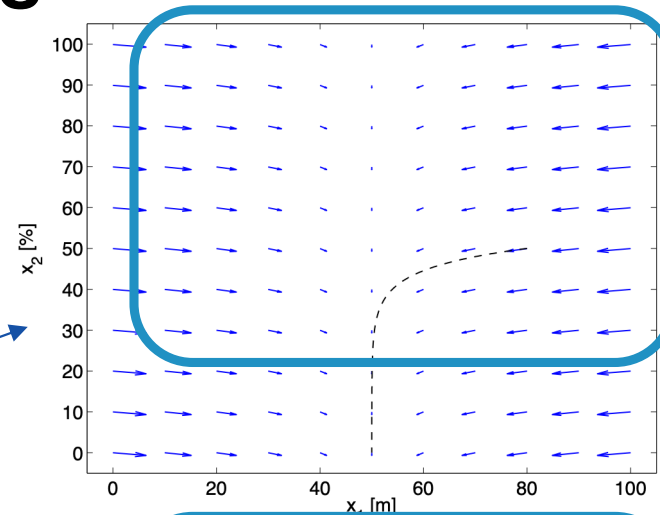
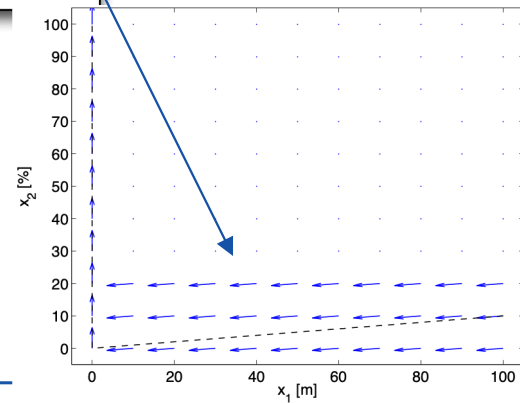
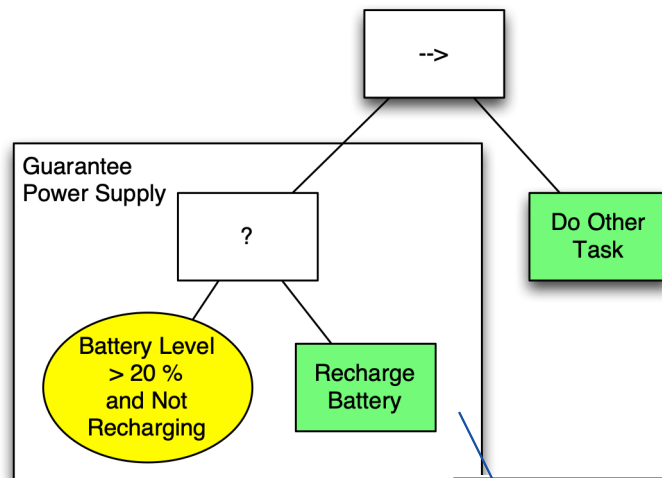


Sometimes we can prove convergence...

# Example: Avoiding Empty Batteries



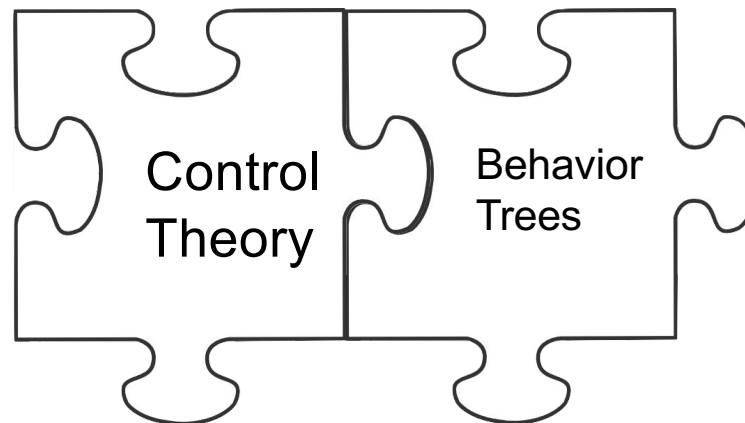
# Avoiding Empty Batteries





# The Big Picture

- Can we give Performance Guarantees?
- Stability/Convergence?



IEEE ROBOTICS AND AUTOMATION LETTERS, VOL. 5, NO. 4, OCTOBER 2020

6073

## Convergence Analysis of Hybrid Control Systems in the Form of Backward Chained Behavior Trees

Petter Ögren

**Abstract**—A robot control system is often composed of a set of low level continuous controllers and a switching policy that decides which of those continuous controllers to apply at each time instant. The switching policy can be either a Finite State Machine (FSM), a Behavior Tree (BT) or some other structure. In previous work we have shown how to create BTs using a backward chained approach that results in a reactive goal directed policy. This policy can be thought of as providing disturbance rejection at the task level in the sense that if a disturbance changes the state in such a way that the currently running continuous controller cannot handle it, the policy will switch to the appropriate continuous controller. In this letter we show how to provide convergence guarantees for such policies.

**Index Terms**—Behavior-based systems, robot safety, control architectures and programming.

### I. INTRODUCTION

**B**EHAVIOR Trees (BTs) were created by computer game programmers as a way to improve modularity and reactivity in the control policies of so-called Non-Player Characters (NPCs) in games [1]. Since then, BTs have been receiving an increasing amount of attention in Robotics [2]–[9]. The reason is that robotics share many high level planning and control problems with game AI, while at the same time, the low

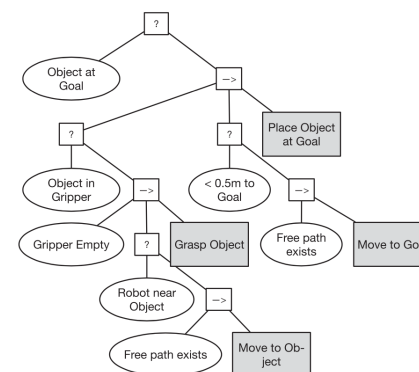
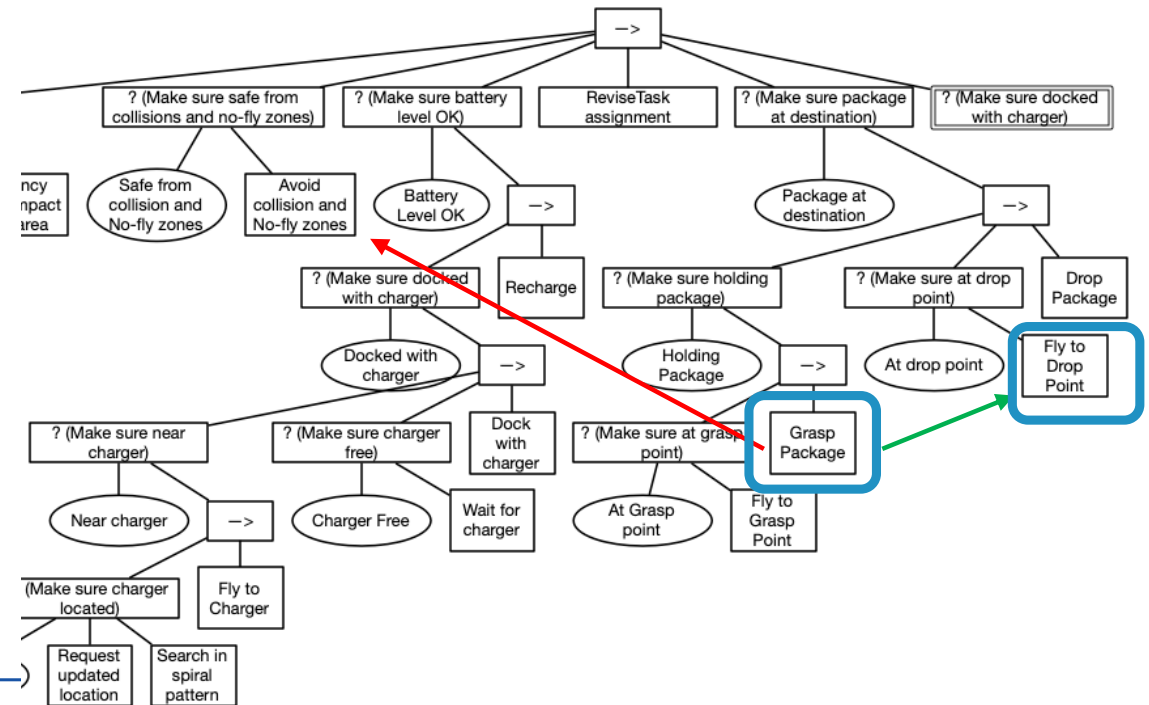
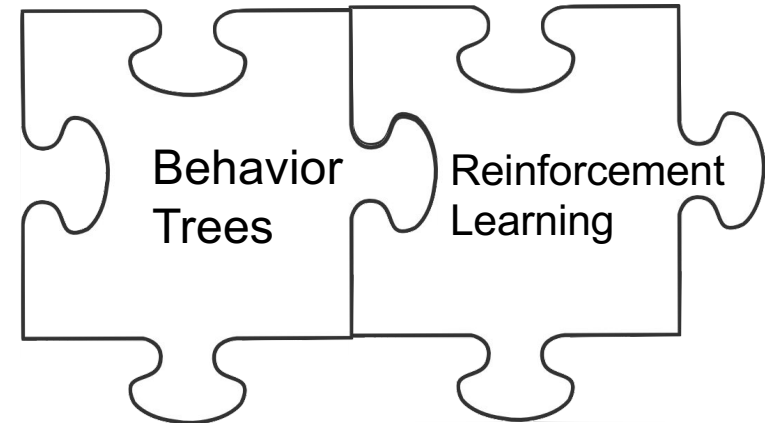
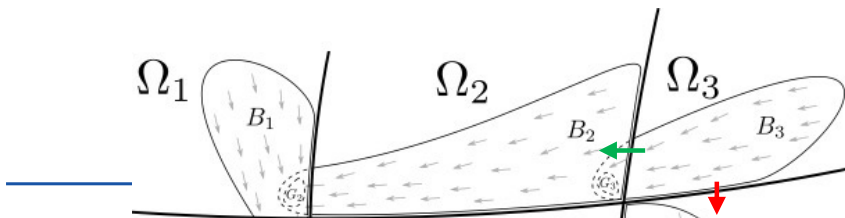


Fig. 1. A BT including the four actions *Move to Object*, *Grasp Object*, *Move to Goal*, *Place Object at Goal*, designed in a way to provide disturbance rejection at the task level. An extended version, including additional objectives and alternative ways to achieve subgoals, can be found in Fig. 5.

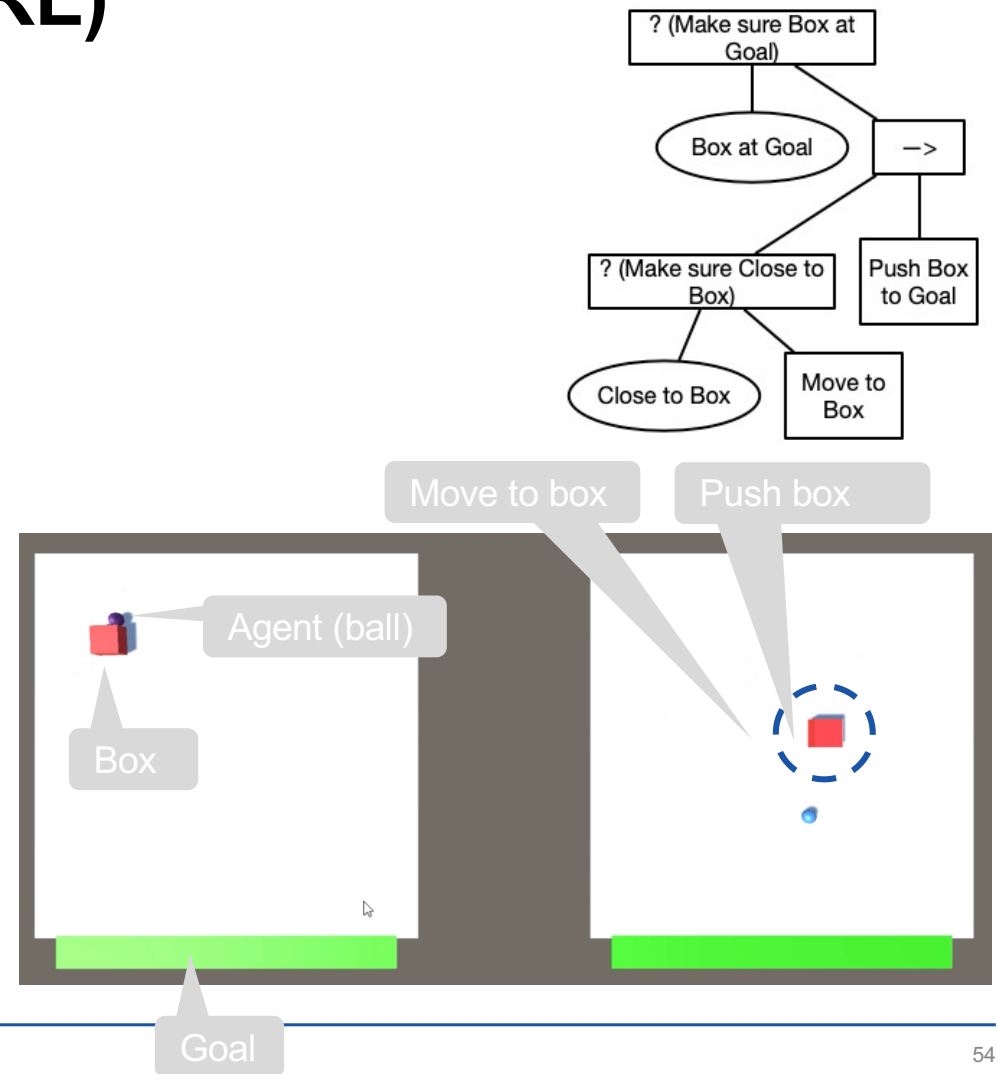
# The Big Picture

- Some tasks are “easy” other “hard”
- Use Reinforcement learning to do hard task
- Get rewards from BT structure
  - Reach post condition
  - Avoid “wrong” switching
- Example of “suboptimality problem”...

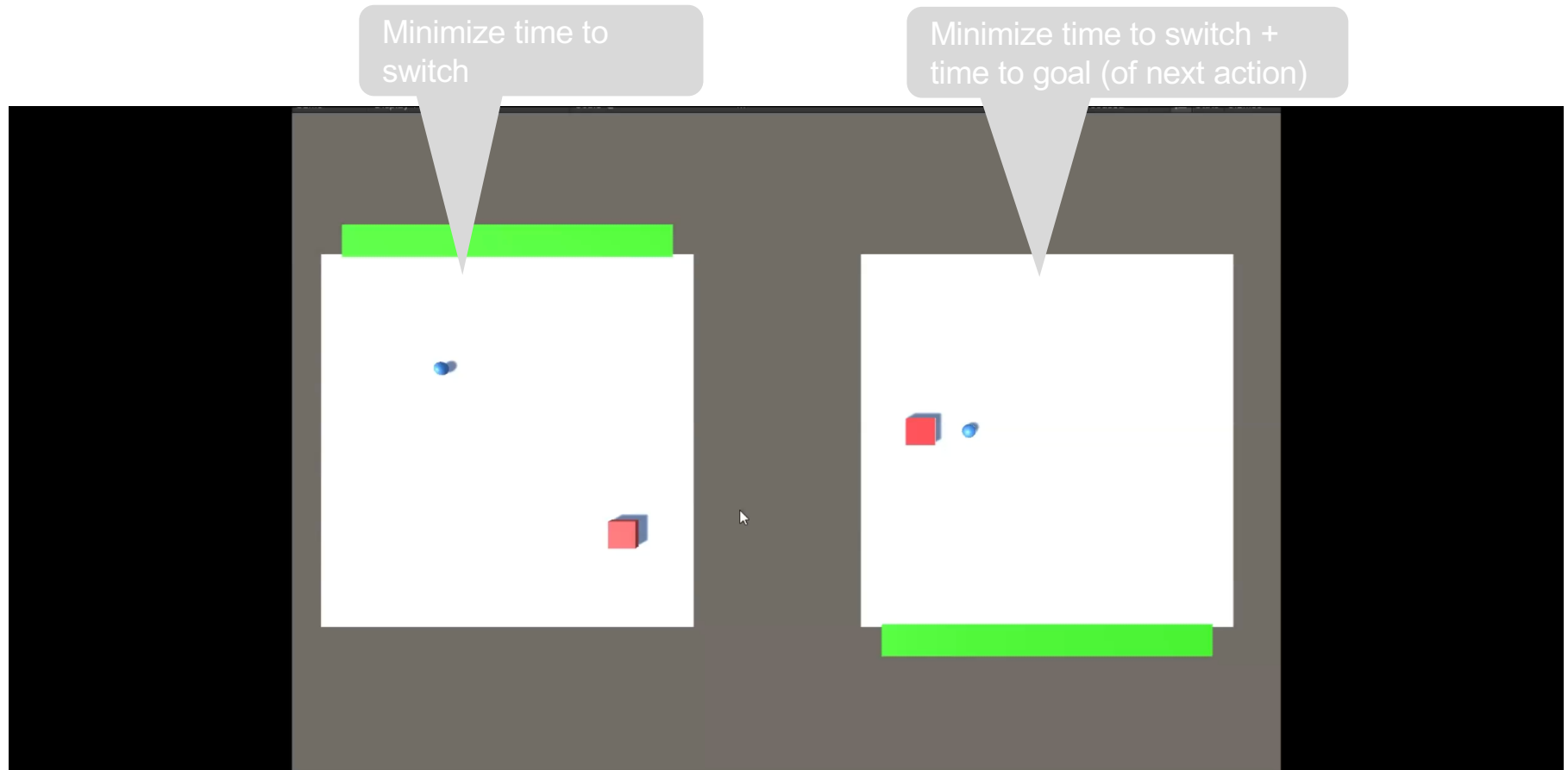


# Pushing a Box (using RL)

- Goal: Box in Goal area
- Behaviors
  - Move to the box
  - Push it to the goal
- Can we train these separately?



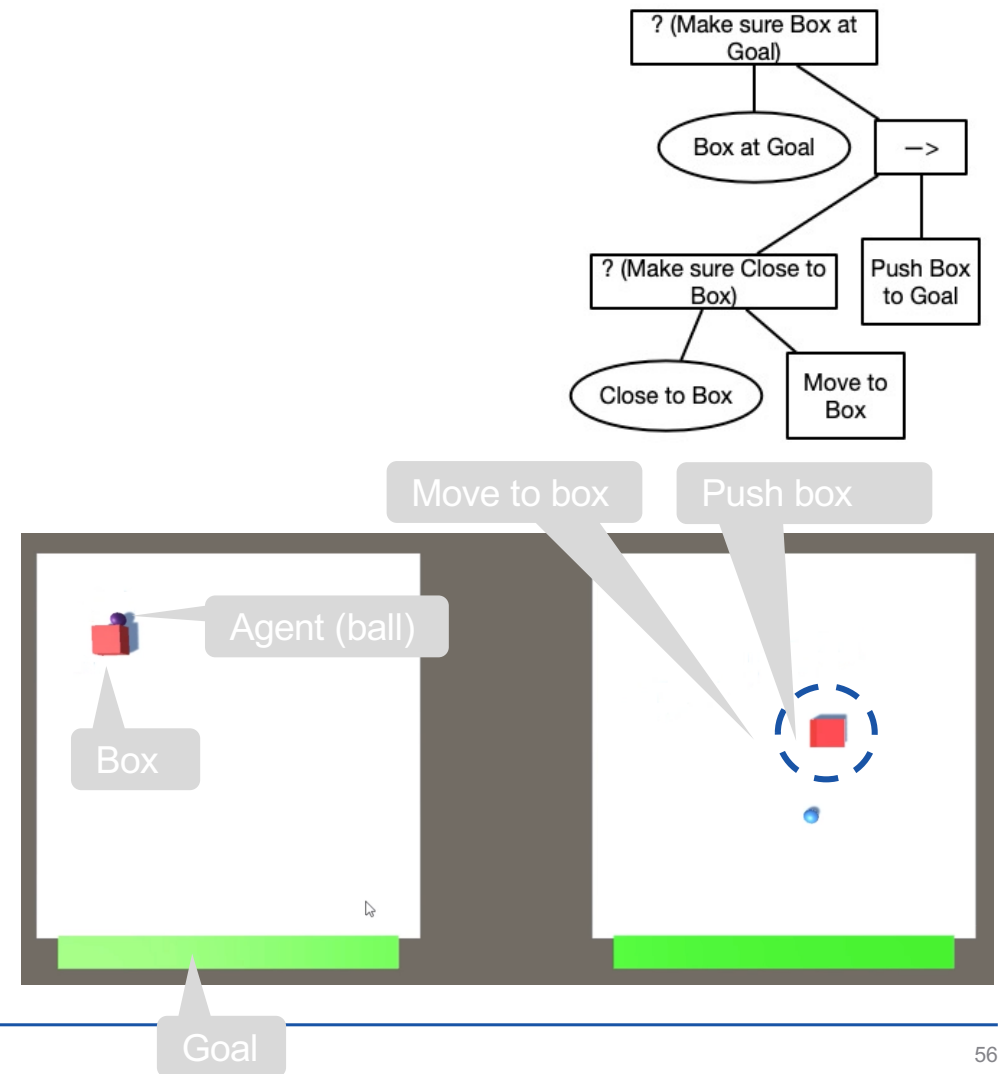
# What is the difference between left/right?





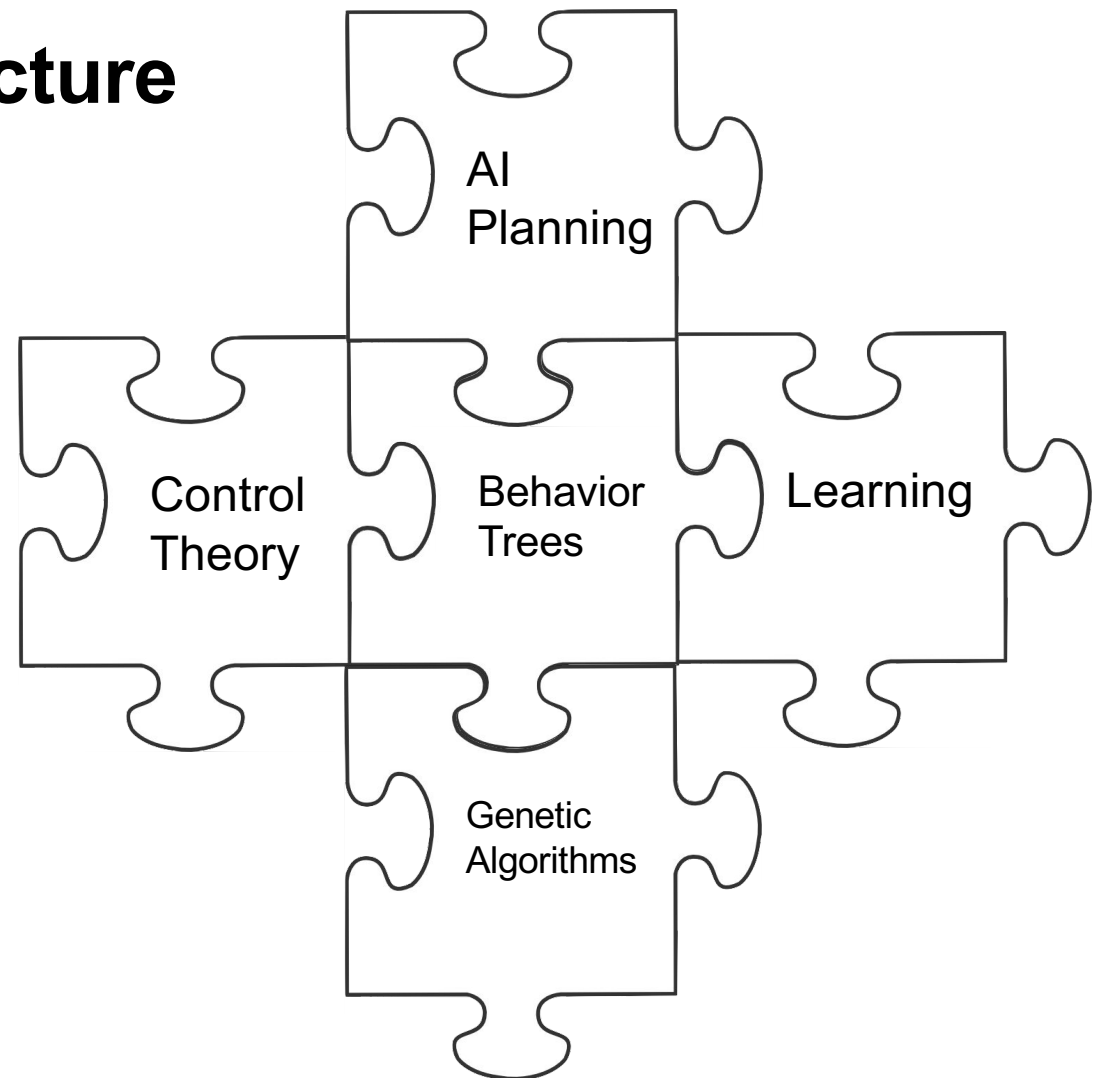
# Key idea

- Use Value function of next policy as final reward for current policy
- Gives Globally optimal combined policy
  - under some assumptions...





# The Big Picture





# Content

- What is a Behavior Tree?
    - A highly modular hierarchical combination of sub-controllers
  - When/Why use a BT?
    - When deciding *what to do next*
    - When you want *hierarchical modularity*
  - How does a BT work? (in detail)
  - How to create BTs?
    - Improvise
    - Use planning (backward chaining of sub-controllers)
  - **The Big Picture**
    - Genetic Algorithms
    - Control Theory (Performance Guarantees)
    - Reinforcement Learning
-



# References

- [1] Biggar, Oliver, Mohammad Zamani, and Iman Shames. "On modularity in reactive control architectures, with an application to formal verification." *ACM Transactions on Cyber-Physical Systems (TCPS)* 6.2 (2022).
- [2] Biggar, Oliver, Mohammad Zamani, and Iman Shames. "An expressiveness hierarchy of Behavior Trees and related architectures." *IEEE Robotics and Automation Letters* 6.3 (2021): 5397-5404.
- [3] Colledanchise, Michele, and Petter Ögren. "How behavior trees modularize hybrid control systems and generalize sequential behavior compositions, the subsumption architecture, and decision trees." *IEEE Transactions on robotics* 33.2 (2016): 372-389.



# Questions?